



Zagazig University
Faculty of Science
Computer Science Department



Graduation project documentation

IOT App Detection Gas With Ai

Project Team :

Mohamed Elsayed Ahmed
Ghada Ebrahim Elsayed
Mariem Nagy Elsayed

Mohamed Lotfi Mohamed
Yara Mohamed Soliman
Youstina Naeem Sobhi

4th Chemistry and Computer Sciences Department

Under the supervision of:

Dr. Hager Ramadan

2024 - 2025

ACKNOWLEDGMENT

It has been our great pleasure to work on this project, we enjoyed the Information Technology field very much to be done with that project it has hard work but we are finding that it also quite fun because we are working under a very good supervisor.

As students of “ **FACULTY OF SCIENCE** ”,
We would like to owe our first appreciation and thanks to “**Dr. Hager Ramadan**” for her care, supervision and for directing us to carry out this project. In addition we would like to express our sincere appreciation to her for her encouragement and support during the project.

At last we would like to thank the supervisors and the university staff, we are grateful for all the help and support they provided to guide us through this project.

However, there were many individuals who have lent a hand, we show gratitude to those who contributed to present our project in the most appropriate manner.

	Chapter 1	
1.1	Introduction	6
1.2	System Workflow	8
1.3	Mobile Application	10
	Chapter 2/ TOOLS	
2.1	Language	15
I.	Html	15
II.	Css	16
III.	Php	17
IV.	Js	18
V.	C++	19
VI.	Dart	20
VII.	Python	21
2.2	Cross Platform	22
I.	Flutter	22
II.	Flask	23
2.3	Database	24
I.	Firebase	24
2.4	Lib	25
I.	Phpmailer	25
II.	Transfer	26
III.	Numy	27
IV.	Firebase_core	28
V.	Firebase_database	29
VI.	Googleapis_auth	30
VII.	Googleapis	31

VIII.	Firebase_messaging	32
IX.	http Flutter_local_notification	33
X.	Fl_chart	34
2.5	Cloud_firestore	35
I.	Intl	35
II.	Animate_do	36
III.	Web_socket_channel	37
IV	Cupertino_icons	38
V	Flutter_launcher_icons	39
2.6	It	40
I.	Ipv4 address	40
2.7	Programs	41
I.	Figma	41
II.	Xampp	43
III.	Vscode	44
IV.	Android Studio	45
V.	Arduino ide	46
2.8	Version Control System	47
I.	Git	47
II.	Service	48
III.	Github	49

	Chapter 3/ Code	
3.1	Database	50
3.2	Flutter	55
3.3	Arduino	68
	CONCLUSION	
	REFERENCES	

Chapter 1

1.1 INTRODUCTION:

This project aims to develop a smart system for detecting gas leaks in water heaters and responding automatically to emergencies, helping to reduce the risks associated with gas leaks and protecting lives and property. The system relies on advanced sensors to detect gas leaks. Upon detection, it automatically shuts off the gas valve, activates a ventilation fan to reduce gas accumulation, and sends instant notifications to the user via a dedicated mobile application, in addition to sending a warning email.

The system is controlled by an electronic control unit that processes data and manages connected devices, such as the electric valve and ventilation system. It also provides remote monitoring capabilities via a smartphone, allowing users to track the heater's status and take necessary actions when needed.

This system is designed for early gas leak detection, minimizing the risk of explosions or suffocation. It can be used in homes, hotels, hospitals, and other facilities that rely on gas water heaters. Additionally, future enhancements may integrate artificial intelligence and machine learning to analyze data and predict leaks before they occur.

By combining real-time response with smart technology, this project contributes to enhancing safety levels in residential and commercial environments, making it an effective and essential solution for public safety.

The goal of the application is:

1. Reduces the risk of fires and explosions caused by gas leaks.
2. Automatic response system eliminates the need for manual intervention in emergencies.
3. Integration with smartphones ensures instant alerts even if the user is not at home.
4. Compact and easy-to-install design that fits inside water heaters without major modification concentration in the air and determines if it exceeds the safe limit.

How does the application work?

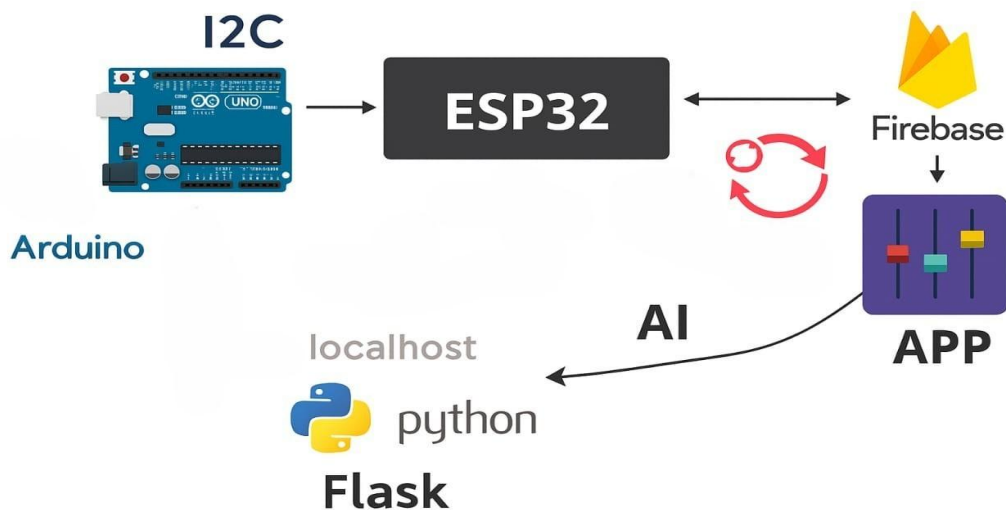
1. **Gas Leak Detection:**
High-sensitivity gas sensors (such as MQ-6 or MQ-9) are installed inside the water heater.
2. When an abnormal gas leak is detected, the sensor analyzes the gas concentration in the air and determines if it exceeds the safe limit.
3. **Sending Notifications to the Mobile Phone:**
Once a leak is detected, the system instantly sends an alert to the user's mobile phone via a dedicated app, email.
4. **Automatic Neutralizing Gas Release:**
An automated mechanism is triggered to release a safe neutralizing gas (such as carbon dioxide or water vapor), reducing the gas concentration and preventing ignition.

Application advantages:

- Instant Emergency Response
- Smart Notifications & Alerts
- Remote Monitoring & Control
- Gas Consumption Optimization
- Enhanced Safety
- Future Scalability

1.2 System Workflow

The system is designed to provide a seamless integration between hardware sensors, cloud services, a mobile application, and an AI-powered decision-making module. Each component in the architecture plays a vital role in achieving real-time data acquisition, processing, and interaction. Below is a detailed breakdown of the system components and their interactions:



1. Arduino Uno (Sensor Data Collection Unit)

The Arduino Uno is responsible for collecting sensor data from the physical environment. These sensors could include temperature, humidity, motion detectors, or any other relevant modules depending on the project requirements. The Arduino processes the raw data and prepares it for transmission via the I2C communication protocol.

2. ESP32 Microcontroller (Data Bridge & Communication Hub)

Acting as the central communication hub, the ESP32 receives sensor data from the Arduino through the I2C interface. The ESP32 is equipped with built-in Wi-Fi capabilities, enabling it to connect to the internet and interact with cloud-based platforms.

- The ESP32 sends the collected data to Firebase, which acts as the backend cloud database for real-time data storage and synchronization.
- It also receives commands or updates from Firebase and forwards them to the Arduino or other actuators, enabling full two-way communication.
- Additionally, the ESP32 is capable of making HTTP requests to a PHP server (e.g., Melair) to perform tasks such as sending automated email notifications based on certain conditions.

3. Firebase (Cloud-Based Real-Time Database)

Firebase serves as the middleware between the ESP32 and the mobile application. It stores the sensor data received from the ESP32 and makes it instantly available to all connected clients. It also allows the mobile application to send control signals or settings back to the ESP32, enabling responsive remote control functionality.

4. Mobile Application (User Interface & Interaction)

The mobile application is connected to Firebase and provides users with a graphical interface to monitor real-time data and control the system remotely.

- Users can view live sensor readings, receive alerts, and modify system behavior.
- The application also sends relevant data to an AI module for further analysis and prediction, ensuring smarter interaction and automation.

5. Python Flask Server (AI Module on Localhost)

A Flask server, built using Python, hosts the AI module which performs advanced data processing and decision-making. The mobile application communicates with this server (via localhost during development) to send current system data and receive back intelligent predictions or classifications.

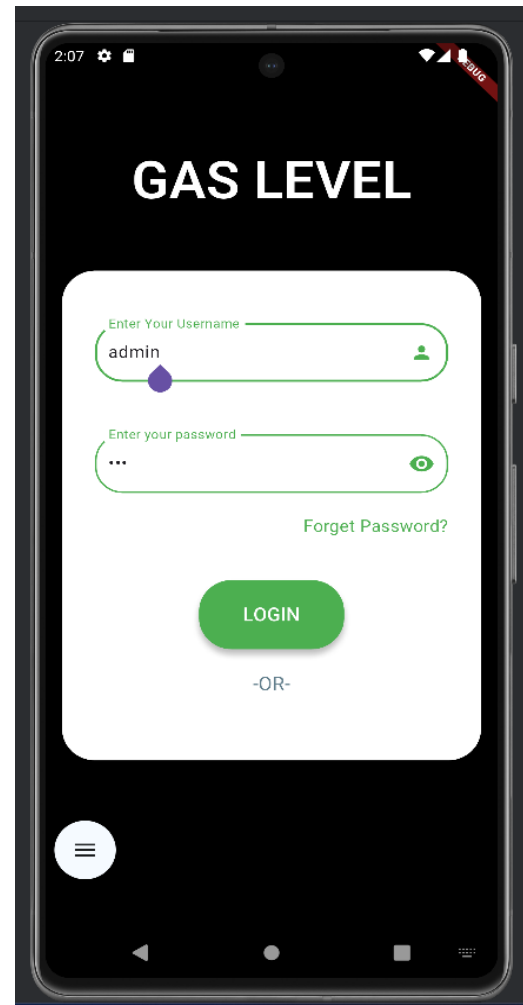
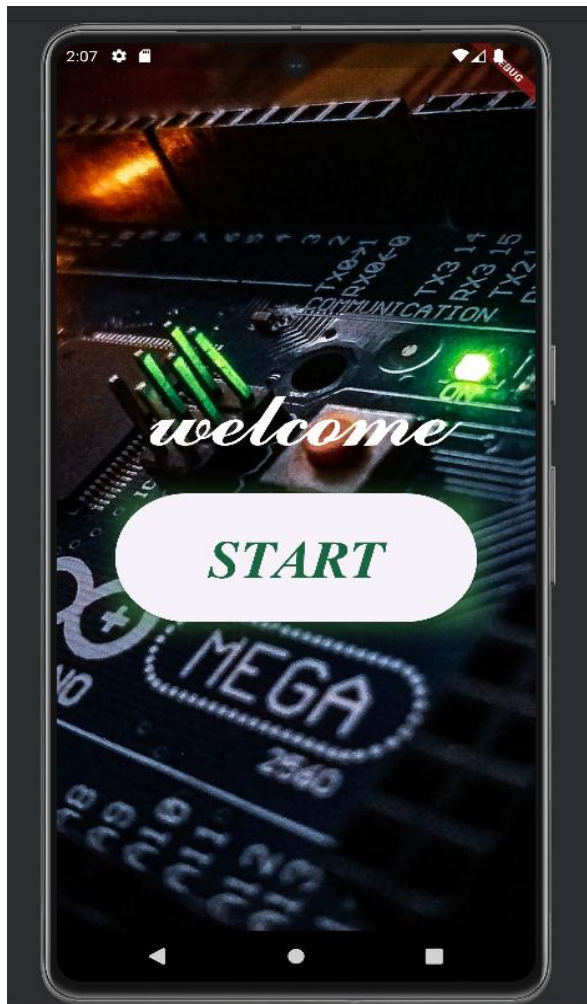
- This AI module can be trained on historical data to predict trends, detect anomalies, or optimize control responses.
- The integration of AI significantly enhances the system's performance by enabling smarter and more autonomous behavior.

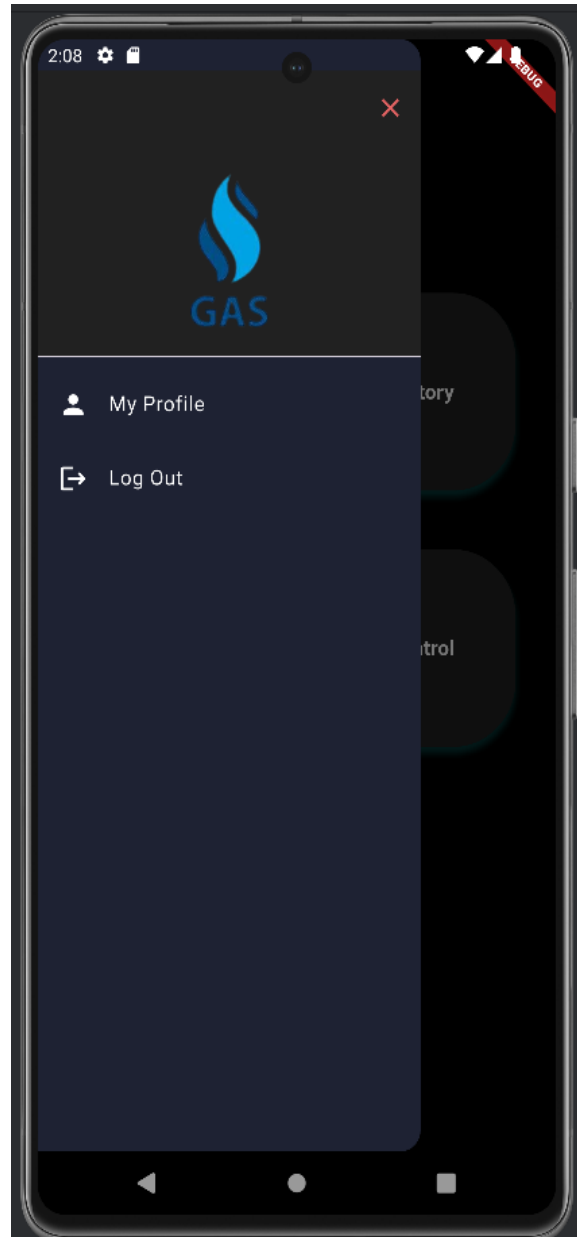
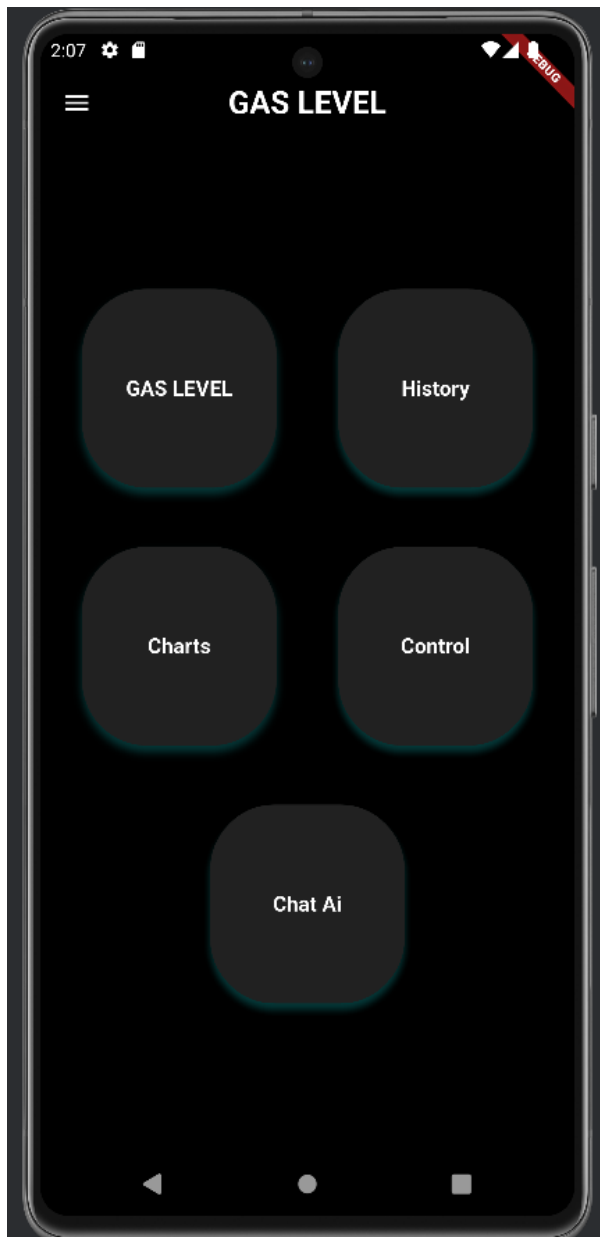
Summary of Data Flow:

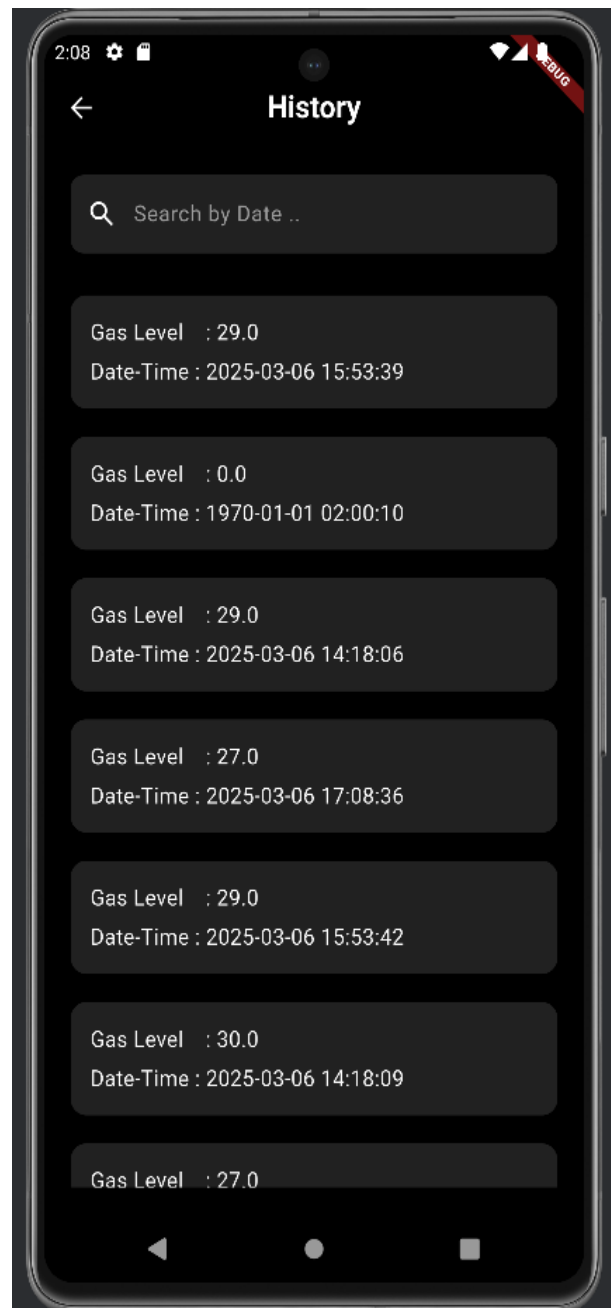
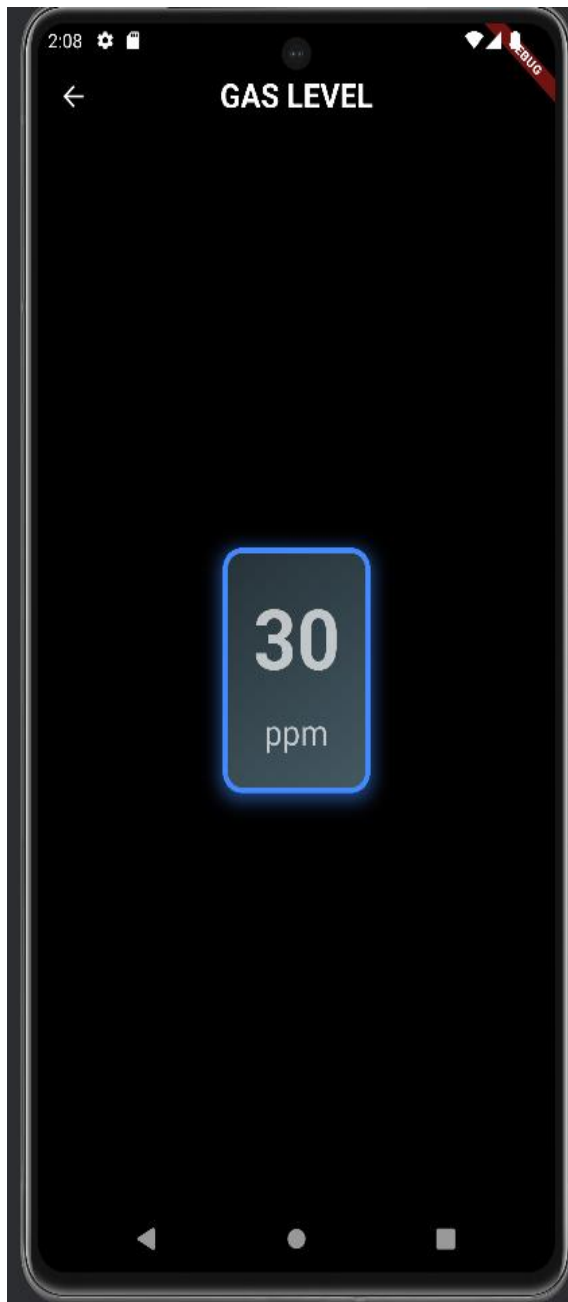
1. Sensor data is collected by the Arduino.
2. Data is transmitted to the ESP32 via I2C.
3. ESP32 sends data to Firebase and, when needed, to a PHP server for email alerts.
4. The mobile app retrieves and displays data from Firebase and allows user interaction.
5. Data from the app is also sent to the Flask-based AI model for further processing and insights.
6. AI results may be used to optimize system behavior or assist users in making better decisions.

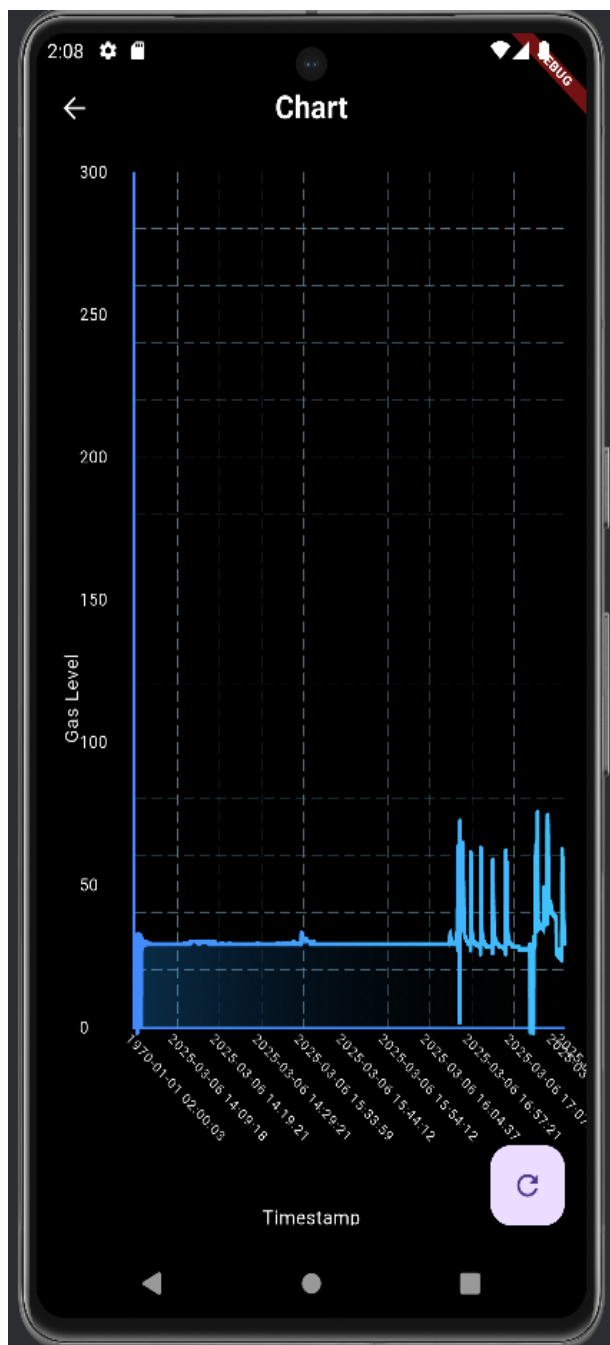
1.3 Mobile Application Screens Overview

The mobile application plays a crucial role in the overall system, acting as the main interface between the user and the smart environment. In this section, we will showcase the different screens of the application, each designed to serve a specific function within the system. These screens are carefully developed to ensure a smooth user experience, real-time data interaction, and full control over the connected components. A brief description will be provided for each screen to explain its layout, purpose, and functionality.









2:08

← Gas

IP

192.168.39.243:1

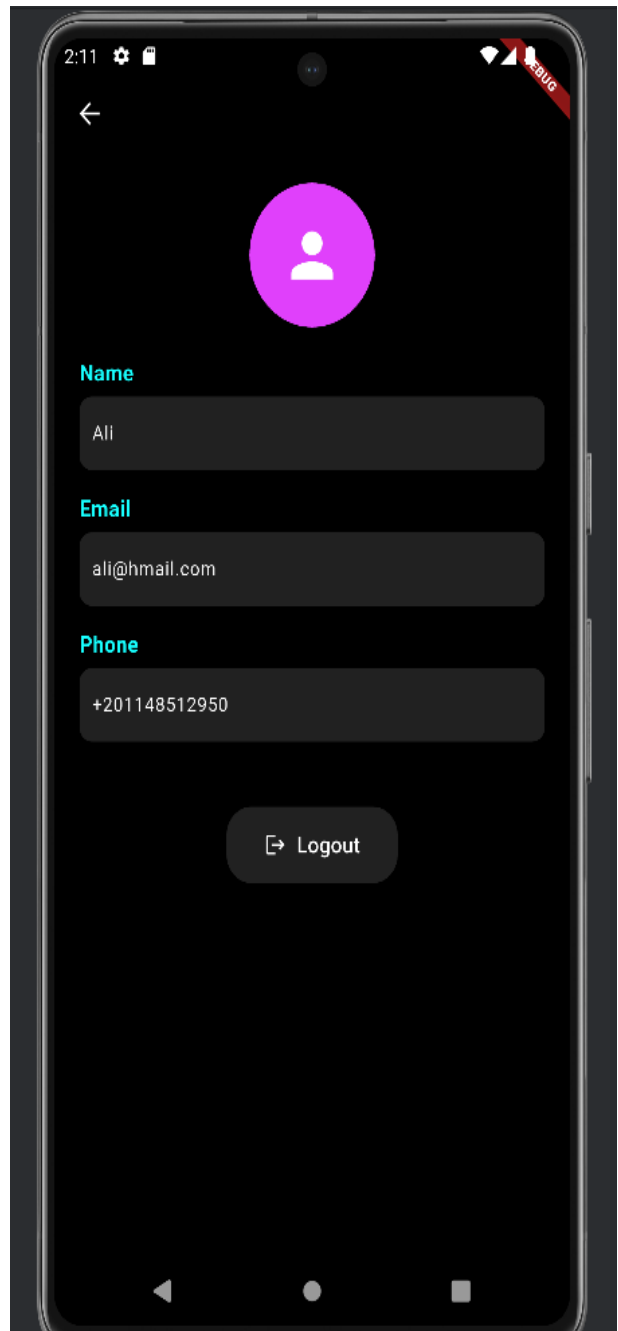
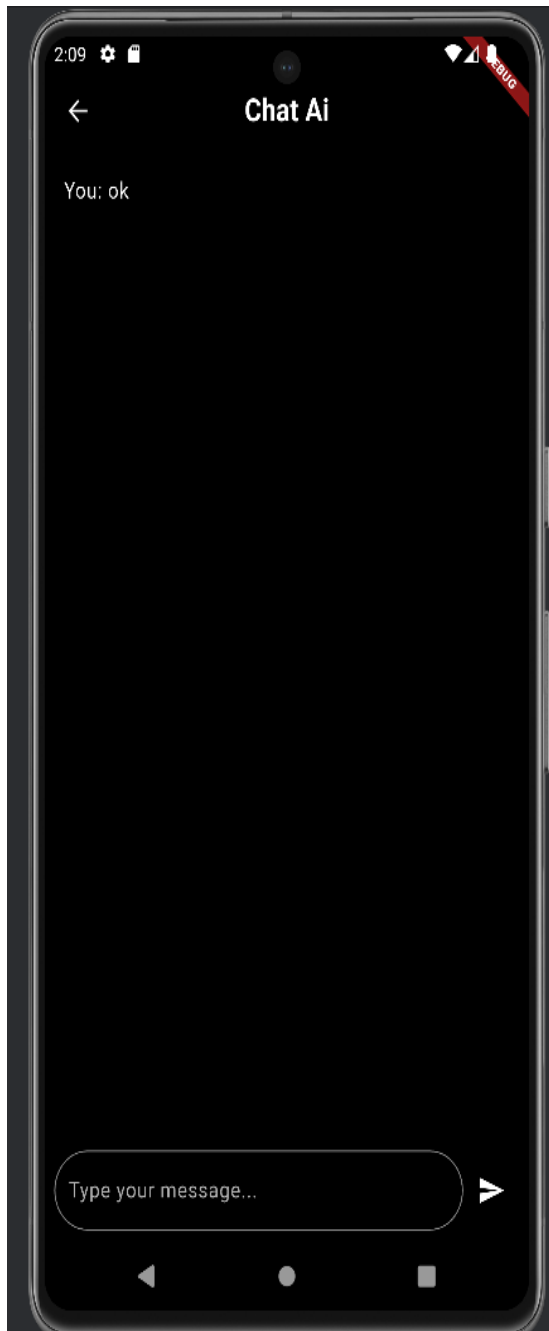
Connect

Not connected

Received:

Heater:

Aug



Chapter 2/ TOOLS

2.1 Language

I. HTML

HTML (Hyper Text Markup Language) is a markup language used to create and structure web pages. It works through a set of tags that define how content is displayed in a browser, such as text, images, links, lists, and tables.



Importance of HTML :

1. Foundation of all web pages: No website can be built without HTML, as it serves as the base for all web development technologies.
2. Easy to learn and use: HTML is simpler compared to other programming languages, making it easy for beginners to learn.
3. Compatible with all browsers: HTML works on all modern web browsers, ensuring consistent content display.
4. Integration with other technologies: HTML can be combined with CSS for styling, JavaScript for interactivity, and PHP or Python for dynamic websites.
5. Search Engine Optimization (SEO) benefits: HTML helps improve website ranking in search engine results by using proper tags like <meta> and <title>.

II. CSS

CSS (Cascading Style Sheets) is a stylesheet language used to control the presentation and layout of HTML documents. It allows developers to style web pages by defining colors, fonts, spacing, positioning, and overall design.



Importance of C++ :

1. Enhances Web Design – Enables customization of fonts, colors, and layouts to create visually appealing websites.
2. Improves User Experience – Helps in designing responsive and user-friendly interfaces.
3. Separation of Content and Design – Keeps HTML structure clean by handling styling separately.
4. Enables Responsive Design – Allows websites to adapt to different screen sizes and devices.
5. Improves Performance – Reduces page load time by using external stylesheets instead of inline styles.

III. PHP

PHP (Hypertext Preprocessor) is a server-side scripting language used primarily for web development. It is embedded in HTML and is widely used to create dynamic and interactive web applications.



Importance of PHP :

1. **Server-Side Processing** – Executes code on the server, generating dynamic content before sending it to the client.
2. **Database Integration** – Works seamlessly with databases like MySQL, making it ideal for building data-driven applications.
3. **Open-Source and Cost-Effective** – Free to use with a vast community for support and development.
4. **Cross-Platform Compatibility** – Runs on various operating systems, including Windows, Linux, and macOS.
5. **Secure and Scalable** – Provides built-in security features and can handle large-scale applications efficiently.

IV. JS

JavaScript (JS) is a powerful, high-level programming language used to create interactive and dynamic web pages. It is a client-side language, meaning it runs directly in the user's browser, enhancing user experience without requiring server communication.



Importance of JavaScript :

1. Interactivity – Enables dynamic elements such as animations, pop-ups, and form validations.
2. Client-Side Execution – Reduces server load by processing tasks directly in the browser.
3. Works with HTML and CSS – Enhances web design by enabling interactive features.
4. Supports Modern Web Development – Used in frameworks like React, Angular, and Vue.js for building advanced web applications.
5. Asynchronous Operations – Uses AJAX and Fetch API to load data without refreshing the page, improving performance.

V. C++

programming language that combines object-oriented, procedural, and generic programming features. It is widely used in software development, game development, system programming, and more.



Importance of C++

1. High Performance – Offers fast execution and efficient memory management, making it ideal for system-level applications.
2. Object-Oriented Programming (OOP) – Supports concepts like inheritance, polymorphism, and encapsulation, improving code reusability and organization.
3. Cross-Platform Compatibility – Can run on multiple operating systems, including Windows, Linux, and macOS.
4. Versatility – Used in various fields such as game development, operating systems, embedded systems, and financial applications.
5. Strong Community and Libraries – Has a rich set of libraries and frameworks, speeding up development.

VI. Dart

Dart is a client-optimized programming language developed by Google, designed for fast apps on any platform. It is an object-oriented, class-based language with a syntax similar to Java or JavaScript. Dart is best known for being the language used to build Flutter apps.



Importance of Dart :

1. **Used with Flutter Framework :** Dart is the core language behind Flutter, a powerful UI toolkit that allows developers to build cross-platform mobile apps from a single codebase.
2. **Fast Performance :** Dart offers Ahead-of-Time (AOT) compilation for native performance on mobile devices, making apps run fast and smoothly.
3. **Hot Reload Feature :** When used with Flutter, Dart supports Hot Reload, allowing developers to see changes in real-time without restarting the whole app. This boosts productivity and speeds up development.
4. **Cross-Platform Development :** Dart allows writing code once and running it on both Android and iOS, which reduces development time and cost.
5. **Easy to Learn & Maintain :** Dart's clean syntax and strong typing make it easy for developers to write, read, and maintain code efficiently in mobile applications.

VII. Paython

Python is a high-level, interpreted programming language known for its simplicity, readability, and versatility. Created by *Guido van Rossum* and first released in 1991, Python has grown to become one of the most popular languages in the world. Its clear syntax and dynamic typing make it an ideal choice for both beginners and experienced developers.

Python supports multiple programming paradigms, including object-oriented, procedural, and functional programming. It has a rich ecosystem of libraries and frameworks that enable rapid development in various fields such as web development, data analysis, artificial intelligence, machine learning, automation, and scientific computing.



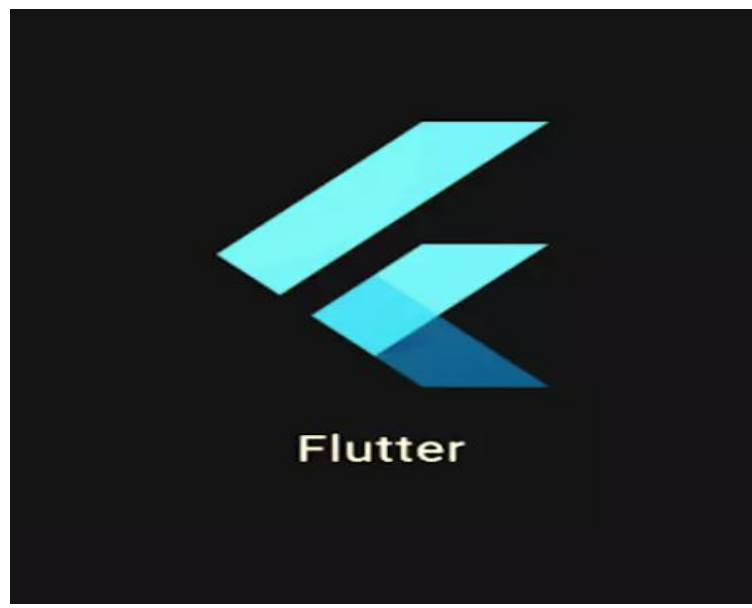
Importance of Python :

1. Easy to Learn & Use – Ideal for beginners due to its readable syntax.
2. Versatile – Can be used for web apps, data science, AI, automation, and more.
3. Large Community – Offers extensive support and resources.
4. Rich Libraries – From NumPy to Django, Python has a tool for almost everything.
5. Cross-Platform – Works seamlessly across different operating systems.

2.2 Cross Platform

I. Flutter

The mobile application, which serves as the user's main interface with the smart gas leak detection system, was developed using Flutter, Google's cross-platform UI toolkit. Flutter was chosen for its ability to deliver high-performance, visually appealing, and consistent applications on both Android and iOS from a single codebase.



Important of Flutter

1. Receive Instant Notifications when a gas leak is detected.
2. Monitor Live System Status, including gas concentration levels and the state of the electric valve and ventilation fan.
3. Access Remote Control Features, allowing users to monitor and manage the system even when they are away from home
4. Enjoy an Intuitive User Interface, designed with Flutter's rich widget library to enhance usability and provide a smooth user experience.

II. Flask

The backend system of the smart gas leak detection project was developed using Flask, a lightweight and flexible web framework for Python. Flask played a key role in managing the communication between the hardware components and the mobile application, as well as handling user data and notifications.



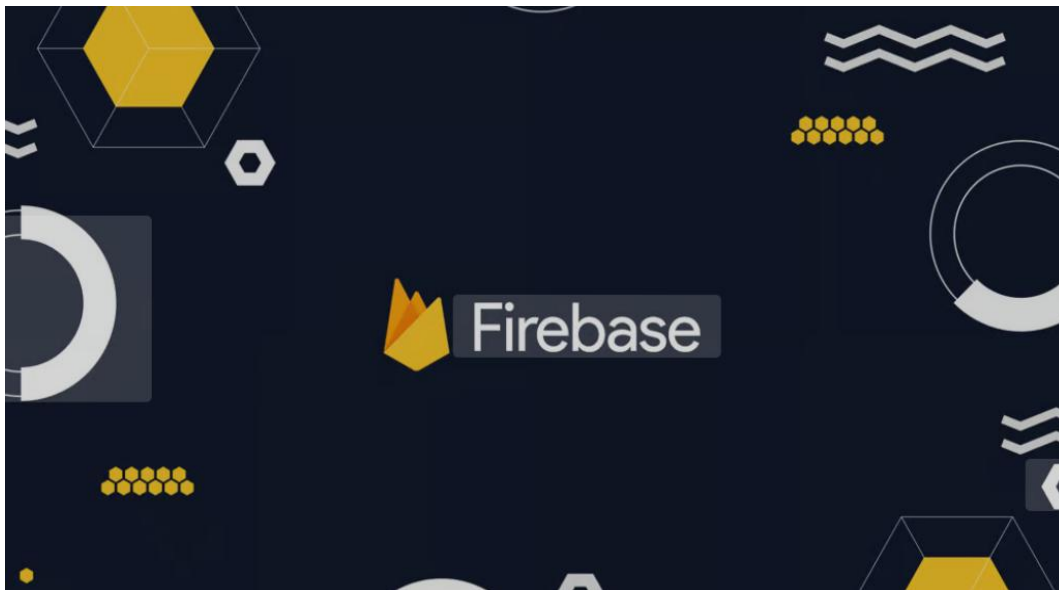
Important Of Falsk:

1. Receive Data from the Gas Sensors and process it in real time.
2. Trigger Notifications and Emails when a gas leak is detected.
3. Store and Serve Data Securely, such as gas levels, timestamps, and system logs.
4. Provide APIs that allow the mobile app (built with Flutter) to fetch real-time data and control the devices remotely.
5. Ensure Secure Communication between the app and the electronic control unit

2.3 Database

I. Firebase

Firebase, Google's powerful Backend-as-a-Service (BaaS) platform, was utilized in this project to support real-time communication, cloud-based data management, and push notifications. Its integration with the mobile app and backend system enhanced the overall responsiveness and scalability of the smart gas leak detection system.



Important of Firebase:

1. Send Real-Time Push Notifications to the user when a gas leak is detected.
2. Store and Sync Live Data from the sensors to the cloud, allowing users to monitor gas concentration levels instantly.
3. Keep Historical Logs for gas leak events and system activity, which can be useful for safety audits and future AI-based predictions.
4. Integrate Seamlessly with Flutter, enabling fast development and efficient data handling on the app side.
5. Ensure Secure Authentication and controlled access (if user login or account features are used).

ZZZZZ2.4 Lib

PHPMailer

To ensure users are promptly informed in case of a gas leak, the system utilizes PHPMailer, a popular and reliable email-sending library for PHP. PHPMailer allows the system to send automatic warning emails to predefined recipients the moment a gas leak is detected.



Important Of PHPMailer:

1. Send Alert Emails instantly when the gas sensor detects dangerous levels
2. Customize Email Content, including detailed information such as gas concentration percentage, time of detection, and device status.
3. Use Secure SMTP Protocols to ensure safe and authenticated email delivery.
4. Enhance Reliability compared to basic PHP mail () function, especially when sending emails from hosting servers.

III.Transfer

The Transfer module is a core part of the system's software architecture, responsible for **handling data exchange** between the microcontroller (hardware), the backend server, and the cloud platform. It ensures that all system events and sensor readings are delivered accurately and efficiently in real time.

Importance of Transfer:

Key Roles of Transfer in the Project:

1. Sensor Data Transfer

- Reads gas concentration values from sensors (e.g., MQ-2, MQ-135) via Analog/I2C/SPI interfaces.
- Converts raw signals into digital data for the microcontroller (Arduino/ESP32).

2. Wireless Alert Transfer

- Sends real-time leak alerts to users via Wi-Fi/MQTT (mobile app) and SMTP (email).
- Enables remote emergency actions (valve shutoff, fan control).

3. Command Transfer

- Receives user commands from the app (e.g., manual shutdown) via Bluetooth/Wi-Fi.
- Executes actions through GPIO pins (valve, fan control).

4. Cloud Data Transfer (Optional)

- Logs sensor readings to databases (Firebase/MySQL) for analytics.
- Supports AI-based leak prediction in future upgrades.

Transfer is Critical:

- Ensures real-time response (milliseconds matter in gas leaks).
- Links hardware to user (bridges physical sensors with digital alerts).
- Enables smart automation (no human intervention needed).

Numy

Numy is a specialized **data transfer and normalization module** in our system, designed to:

- **Standardize sensor readings** (e.g., convert analog signals to unified digital values)
- **Bridge hardware-software communication** (microcontroller ↔ mobile app/cloud)
- **Optimize data packets** for low-bandwidth wireless transmission

Key Functions in Your Project

1. Signal Normalization

- Converts raw ADC (Analog-to-Digital) readings from gas sensors (MQ-2/MQ-5) into **calibrated ppm values**
- Applies **noise reduction** to minimize false alarms

2. Data Packaging

- Compresses sensor data into efficient formats before wireless transmission (Wi-Fi/Bluetooth)
- Ensures compatibility between:
 - **Microcontroller output** (e.g., ESP32)
 - **Mobile app input** (e.g., Flutter/React Native)

3. Protocol Translation

- Translates between:
 - Hardware-level protocols (I2C/SPI for sensors)
 - Application-level protocols (MQTT/HTTP for alerts)

Numy is Critical:

- Ensures Data Accuracy & Consistency
- Enables Real-Time Performance
- Future-Proofs the Architecture

Firestore_Core

Firestore Core is the foundational package that enables:

- Initialization of all Firestore services in your app
- Connection between your app and your Firestore project
- Unified configuration for all Firestore features

Important Of Firestore_Core:

1. Instant Alerts When Every Second Counts

- Push Notifications in <1 Sec → Users get immediate leak warnings on their phones.
- No Delays → Firestore Core pre-initializes all services so alerts fire instantly when sensors detect danger.
- Works Even Offline → Stores alerts if the internet drops, then sends them when back online.

2. Keeps Your Hardware & Software in Sync

- ESP32 Microcontroller? Talks to Firestore Core.
- Android/iOS App? Syncs in real-time.
- Admin Dashboard? Gets live updates.
- No More "Signal Lost" Errors → Handles bad connections automatically.

3. Makes Scaling to 1000s of Devices Easy

- From 1 home to entire cities → Firestore Core manages all devices without slowing down.
- New Features? Just plug them in → Core automatically connects them to your existing system.

4. Bulletproof Security (No Hackers Allowed!)

- Encrypts every alert so hackers can't fake gas leaks.
- Checks device IDs → Only YOUR sensors can send commands.
- GDPR-ready → Keeps user data 100% private.

v. Firebase_Database

Firebase Database (Firestore/Realtime DB) is Google's cloud-hosted NoSQL database that:

- Stores all your gas sensor readings, alerts, and device statuses
- Syncs data instantly across mobile apps, admin dashboards, and IoT devices
- Works offline-first - queues data if internet drops, then syncs when reconnected

Important Of Firebase Database:

1. Life-Saving Real-Time Sync

- When sensors detect gas → DB updates <1 second to:
 - Mobile apps (immediate alerts)
 - Admin dashboards (live monitoring)
 - Hardware (auto-shutdown commands)

2. Complete Incident History

- Essential for post-incident analysis and system improvements

3. Military-Grade Security

- Rules system ensures:
 - Only authorized devices can write sensor data
 - Homeowners see only their property's data
 - All data encrypted in transit and at rest

4. Hardware Control Hub

- **Your ESP32 devices:**
 - Read shutdown commands
 - Confirm actions ("Valve closed successfully")
 - Sync status every 5 seconds (failsafe checks)

5. Global Scalability

- Handles:
 - 1 home (basic plan) → 10,000+ devices (enterprise)
 - Automatic load balancing during mass alerts

vi. Googleapis_Auth

The `googleapis_auth` package is a Dart library that enables secure authentication and authorization when accessing Google APIs. It handles the OAuth 2.0 authentication flow and allows applications to interact with various Google services in a secure and reliable manner.

Important Of Googleapis_Auth :

1. Secure Gmail Access:

Authenticates the system to securely access the Gmail API using OAuth 2.0, ensuring that email alerts are sent through a verified and protected connection.

2. Automated Emergency Notifications:

Allows the system to automatically send alert emails to the user when a gas leak is detected, providing real-time response without manual intervention.

3. Data Privacy and Security Compliance:

Ensures that all communications with Google services follow modern security standards, protecting user information and maintaining system integrity.

4. Reliable Integration with Google Services:

Facilitates smooth and reliable communication with Google APIs, which enhances the robustness of the system's alerting mechanism.

vii. Googleapis

The googleapis package is an official Dart library that provides easy access to a wide range of Google services through their APIs, such as Gmail, Drive, Calendar, and more. It includes pre-built client classes for each Google API, allowing developers to interact with these services without needing to manually construct HTTP requests.

Important Of Googleapis:

1. Direct Integration with Gmail API:

Provides built-in support for sending emails via the Gmail API, allowing the system to notify users instantly when a gas leak is detected.

2. Simplified API Usage:

Offers pre-defined classes and methods for various Google services, making it easier to implement email sending features without dealing with low-level HTTP requests.

3.

4. Enhanced Reliability:

Ensures stable and consistent communication with Google's servers, which is critical for timely delivery of warning messages.

5. Seamless Compatibility with Authentication Tools:

Works seamlessly with the googleapis_auth package to handle secure access and authorization, forming a complete solution for authenticated communication.

viii. Firebase messaging

The `firebase_messaging` package is a Flutter plugin that enables communication between your mobile app and Firebase Cloud Messaging (FCM). It allows you to send and receive push notifications across Android and iOS platforms, even when the app is in the background or closed.

Important Of `firebase_messaging`:

1. Real-Time Push Notifications:

Sends immediate alerts to the user's mobile device when a gas leak is detected, even if the app is not open, ensuring critical warnings are always received.

2. Cross-Platform Support:

Provides consistent notification delivery across both Android and iOS, which is essential for reaching users regardless of their device type.

3. Seamless Integration with Firebase:

Works perfectly with other Firebase services, such as Firebase Cloud Functions and Firestore, allowing scalable and responsive system behavior.

4. Enhanced User Engagement and Safety:

Ensures users are instantly aware of emergency situations, increasing safety and response time even when they're away from home.

5. Customizable Notification Payloads:

Supports sending custom data in notifications, enabling advanced features like directing the user to a specific screen or displaying detailed alert information.

ix. 2.4.9 http Flutter_local_notification

The flutter_local_notifications package is a Flutter plugin that allows apps to display local notifications to users on Android, iOS, and other platforms. It gives fine-grained control over how notifications look, behave, and when they appear — even when the device is offline or the app is running in the foreground.

Important Of http Flutter_local_notification:

1. Instant In-App Alerts:

Displays notifications immediately when a gas leak is detected, especially useful when the app is running in the foreground (where Firebase may not show push notifications by default).

2. Offline Notification Support:

Allows the system to notify users even without internet access, by scheduling or triggering local alerts directly on the device.

3. Custom Notification Appearance:

Enables customization of notification sounds, icons, vibration patterns, and priority — ensuring that emergency alerts are noticeable and taken seriously.

4. Scheduled Notifications (Optional):

Can be used for periodic safety reminders or maintenance alerts in future versions of the system.

5. Complements Firebase Messaging:

Works alongside firebase_messaging to cover all notification use cases: background, foreground, and offline, providing a complete and reliable alerting experience.

x. FL_Chart

The fl_chart package is a powerful and flexible Flutter library used to create beautiful and interactive charts such as line charts, bar charts, pie charts, and more. It helps in visualizing data in a clean and modern way, making complex information easier to understand for users.

Important Of FL_Chart:

1. Real-Time Data Visualization:

Allows users to view live updates of gas concentration trends, making it easier to monitor the environment.

2. User-Friendly Interface:

Presents data in a clean and intuitive format, helping users (even non-technical ones) to understand safety levels at a glance.

3. Supports Multiple Chart Types:

Enables the use of various chart types (e.g., line charts for trends, bar charts for comparisons) depending on the nature of the data being displayed.

4. Interactive Features:

Allows panning, zooming, and tooltips for deeper analysis, which can be helpful for advanced users or admins reviewing incident logs.

2.5 Cloud Firestore

I. Intl

The Intl package ensures all system alerts, logs, and UI elements adapt to the user's language, region, and cultural preferences, critical for:

- **Emergency Clarity:** Gas leak warnings must be instantly understandable (e.g., Arabic numerals for MENA users).
- **Regulatory Compliance:** Some countries mandate localized safety notifications.

Important Of Intl:

1. Safety Clarity

- Displays emergency alerts (like gas leaks) in the user's language (Arabic, English, etc.) and local time/date format (e.g., 24/10/2023 vs 10/24/2023).

2. Professional Compliance

- Meets legal requirements in some regions where safety warnings must be in the official language.

3. Global Readiness

- Supports multiple languages & regions (crucial for hotels/hospitals with international users).
- Handles RTL languages (Arabic/Hebrew) correctly.

4. Clean Data for AI

- Ensures logs and reports use consistent formats, helping future AI leak predictions.

II. Animate_do

Animate_do (or react-animate-do/animate.css for web) is a popular animation library that provides pre-built, eye-catching animations for UI elements. It's commonly used in:

- Flutter (via packages like animate_do)
- JavaScript/React (via react-animate.css or framer-motion)
- Web (via animate.css)

Important Of Animate_do:

1. Emergency Attention-Grabbing

Purpose: Animations (e.g., flashes, shakes, pulses) make alerts unmissable during gas leaks.

2. User Experience (UX) Enhancement

Guides users to critical actions (e.g., "Shut Off Valve" button with a bounce effect).

Visual feedback for system status (e.g., fan activation with a spin animation).

3. Professional Polish

Smooth transitions (e.g., fade-ins for alerts) make the app feel high-quality and trustworthy.

4. Cross-Platform Consistency

Works on mobile (Flutter), web (React), and embedded dashboards.

III. Web_socket_channel

A `web_socket_channel` (or `WebSocket` API) enables real-time, bidirectional communication between a client (e.g., your mobile app/web dashboard) and a server. Unlike HTTP (which is request-response), WebSockets maintain a persistent connection, allowing instant data exchange.

Important Of Web_socket_channel:

1. Real-Time Emergency Alerts

- When a gas leak is detected, the server instantly pushes alerts to all connected devices (no refresh needed).

2. Live Device Control

- Users can **remotely shut off valves** or activate fans via the app, with commands sent over WebSocket for **immediate execution**.

3. Data Synchronization

- Syncs sensor data (e.g., gas levels) across multiple admin dashboards **in real time**.

4. Low Latency & Efficiency

- Uses a single, persistent connection (no repeated HTTP overhead), crucial for **life-saving rapid response**.

5. Offline Handling

- Libraries like `web_socket_channel` (Flutter) automatically **reconnect if the network drops**, ensuring reliability.

IV. Cupertino_icons

Cupertino Icons are a set of pre-designed, iOS-style icons provided by Flutter's `cupertino_icons` package. They follow Apple's Human Interface Guidelines, offering a sleek, familiar look for iOS users.

Important Of Cupertino_icons:

1. iOS Consistency

- Provides **native iOS aesthetics** (e.g., back buttons, settings icons) for a polished UX.
- Example: Using `CupertinoIcons.flame` for gas-related alerts matches iOS design language.

2. Cross-Platform Readiness

- Works alongside Material Icons (Android) for **seamless multi-platform apps**.

3. Professional Trust

- Familiar icons reduce user learning curves, critical for emergency apps where clarity saves lives.

v. Flutter_launcher_icons

flutter_launcher_icons is a Flutter package that automatically generates and sets app icons for both Android and iOS from a single image file. It replaces the manual process of resizing and placing icons in multiple platform-specific folders.

Important Of Flutter_launcher_icons

1. Professional Branding

- Ensures your app has a **polished, recognizable icon** on home screens (critical for trust in safety apps).

2. Time-Saving Automation

- Generates **all required icon sizes** (from 48x48 to 1024x1024) with **one command**, eliminating manual work.

3. Platform Consistency

- Maintains **identical icons** across Android and iOS, avoiding platform-specific discrepancies.

4. Easy Updates

- Change your app icon once, and the package **updates all variants automatically**.

2.6 It

I. Ipv4 address

An IPv4 (Internet Protocol version 4) address is a unique numerical identifier assigned to every device connected to a network (e.g., your gas leak detection system's server, sensors, or user apps). It consists of four octets (e.g., 192.168.1.1) and enables devices to communicate over the internet or local networks.

Important Of Ipv4 address:

1. Device Communication

- Allows your sensors, valves, and mobile app to exchange data in real time.
 - Example: A gas sensor (192.168.1.10) sends alerts to the server (192.168.1.1).

2. Remote Monitoring & Control

- Users/admins can access the system remotely via the server's public IPv4 address (e.g., to shut off valves from a phone).

3. Network Reliability

- IPv4 is universally supported by IoT devices, routers, and cloud platforms (unlike IPv6, which isn't always compatible).

4. Emergency Notifications

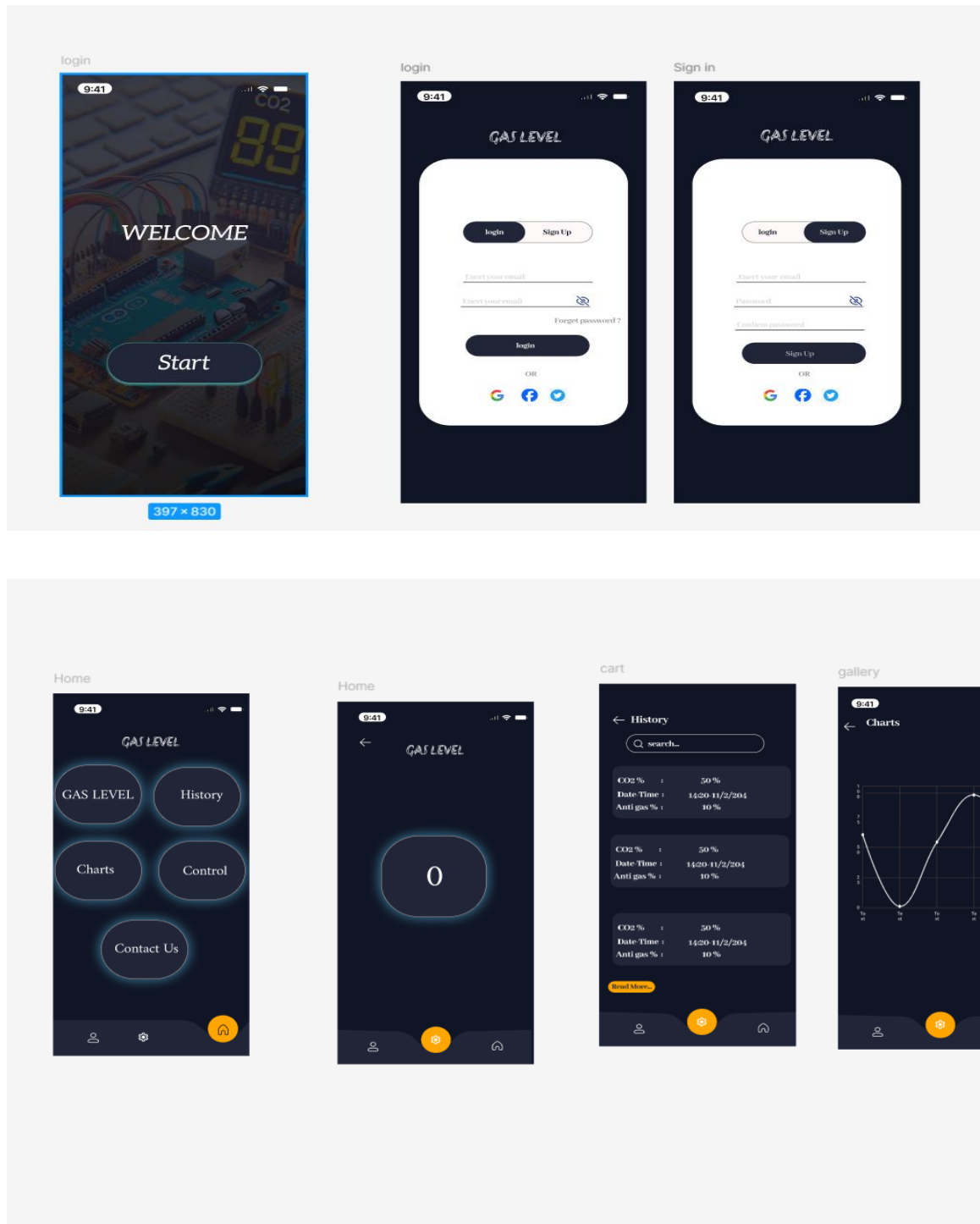
- Your server uses its IPv4 address to send alerts to users' apps/emails via APIs (e.g., Twilio, Firebase).

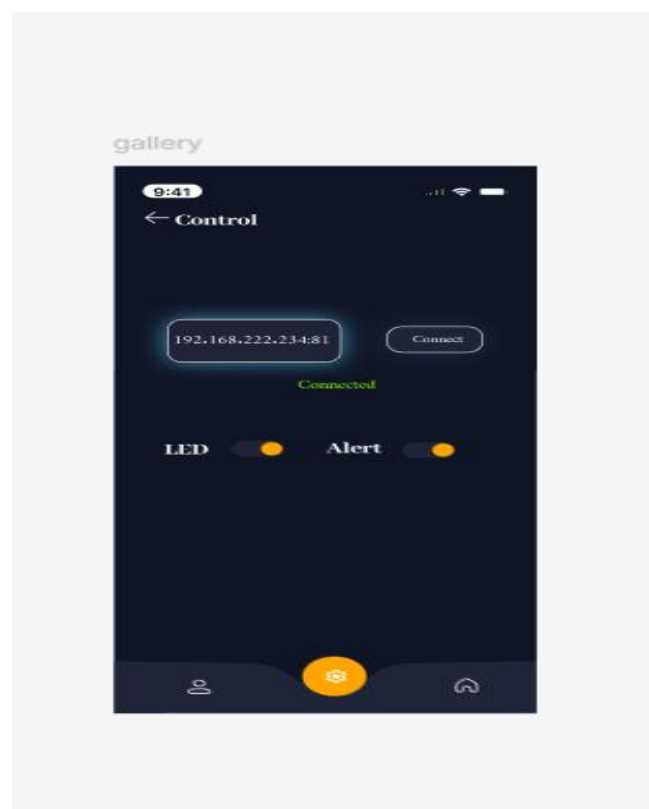
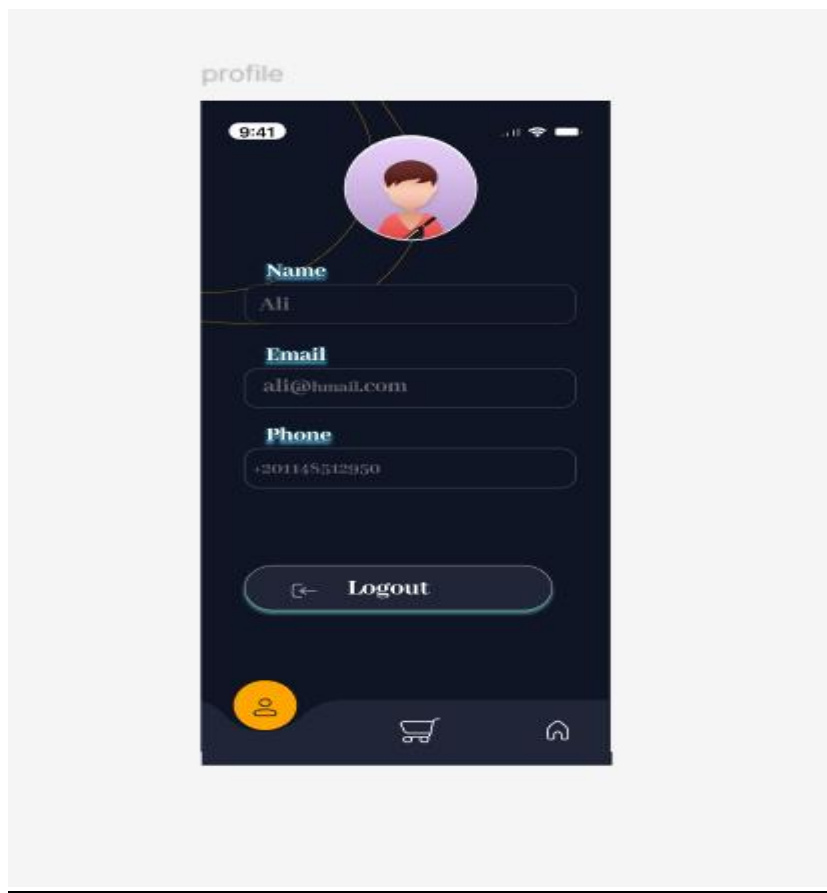
2.7 Progams

I. Figma

I used Figma to design all the UI/UX components for my gas leak detection mobile application.

On the next page, I will display the design screens for the application.





II. Xampp

III. Xampp

A free local server package that lets you run a full web development environment on your computer (Windows/Mac/Linux).



Important Of Xampp:

1. Test & Develop Offline

- Run PHP/MySQL projects on localhost without paying for hosting.

2. Built-in Database

- Store gas leak logs, user data, and device info in MySQL (managed via phpMyAdmin).

3. IoT Prototyping

- Simulate sensor-to-server communication locally before deploying to the cloud.

4. Zero Internet Needed

- Develop and debug completely offline.

v. Vscode

VSCode is a free and open-source code editor developed by Microsoft. It is widely used by developers of all levels for its powerful features, extensibility, and user-friendly interface. VSCode offers support for various programming languages and frameworks, making it a versatile tool for any development project.



Strengths:

- User-friendly interface: Easy to learn and navigate, with a customizable layout.
- Lightweight and efficient: Uses few resources and runs smoothly on various systems.
- Extensive extension library: Thousands of extensions are available for various functionalities and languages.
- Integrated terminal: Provides a built-in terminal for command-line tasks.
- Git integration: Supports Git version control directly within the editor.
- Debugging tools: Offers built-in debugging functionalities for different languages.
- Syntax highlighting and code completion: Enhances code readability and writing efficiency.
- Multilingual support: Supports multiple languages for the interface and documentation

Download Link:

<https://code.visualstudio.com/download>

IV. Android Studio

Android Studio is the official integrated development environment (IDE) for Android app development. Built on JetBrains' IntelliJ IDEA platform, it provides a comprehensive set of tools and features specifically designed for Android developers to build modern and high-quality mobile applications.

Most Popular Features:

- **Layout Editor:** Visually design and build app layouts using a drag-and-drop interface.
- **Code Editor:** Powerful and customizable code editor with syntax highlighting, code completion, and code refactoring tools.
- **Emulator and Device Support:** Run and test your app on a variety of virtual and physical devices.
- **Debugging Tools:** Identify and fix bugs with ease using powerful debugging tools and features.
- **Build Tools:** Build and deploy your app to various channels like the Google Play Store.
- **Support for Multiple Programming Languages:** Develop your app using Java, Kotlin, C++, and other languages.
- **Gradle Support:** Build automation tool for managing dependencies and build configurations.
- **Version Control Integration:** Integrate with version control systems like Git for better collaboration.
- **Large and Active Community:** Access extensive documentation, tutorials, and support from the community.

Download Link:

<https://developer.android.com/studio>

Arduino_ide

Arduino is an open-source electronics platform based on easy-to-use hardware and software. It's widely used for creating interactive electronic projects, offering a variety of boards and components for beginners and advanced users alike.



Strengths:

- **Simplicity:** Arduino makes it easy to get started with electronics using a simple programming language based on C/C++, and a user-friendly development environment (IDE).
- **Wide Community Support:** Arduino has a large and active global community, providing a vast number of tutorials, libraries, and forums for troubleshooting and learning.
- **Flexibility:** Compatible with numerous sensors, modules, and shields, allowing for endless possibilities in creating projects from home automation to robotics.
- **Customization:** Provides flexibility with customizable options for headers, query parameters, request and response interceptors, and more.
- **Cost-effective:** Arduino boards are relatively inexpensive, making it ideal for students, hobbyists, and educators.
- **Cross-Platform:** The Arduino software runs on Windows, macOS, and Linux, offering flexibility in development.
- **Open-source:** Both the hardware and software are open-source, encouraging innovation, sharing, and customization.

Download Link:

<https://www.arduino.cc/en/software>

2.8 Version Control System

I. Git

Git is a copy management system that allows you to save files and keep track of their different copies, whether these files belong to one person or several people involved in the files. For example: let's say you have a file containing a line of text, and you save that file in Git and then add two more lines to the file and save it again, Git will let you go back to the file's first state when it only had one line and it will also let you go once Other to last modified, and as long as you save all changes in Git all the copies you made will be available to you whenever you want.



Benefits of Git:

- Roll back to previous versions to avoid any errors during work.
- Keep track of all copies that have been worked freely.
- Git allows team members to simultaneously work on the same files so that everyone on the team has their copy.
- The possibility of creating branches (copies) of the original 15 work files, developing them independently, testing them without affecting the original file, and then merging them with it.

Download link:

<https://git-scm.com/downloads>

II. A Service

A Service is a function or component that performs a specific task, usually in the background or as part of a larger system, without requiring direct interaction from the user.

Strengths:

- **Background Task Execution:** Services allow critical processes (like sensor monitoring and emergency response) to run in the background, even when the app is closed or minimized.
- **Real-Time Notifications:** Services work with tools like Firebase Messaging and local notifications to ensure that users receive instant alerts about gas leaks or system updates.
- **System Automation :**They automate key actions such as shutting off the gas valve, turning on the ventilation fan, and sending alert emails without user intervention.
- **Seamless User Experience:** By running operations behind the scenes, services maintain a smooth and uninterrupted user interface, enhancing usability and responsiveness.
- **Improved Safety and Response Time:** Services help the system react instantly to emergencies, reducing the risk of accidents and improving overall safety for users.
- **Scalability for Future Enhancements:** The service-based structure allows easy integration of future features like AI-based leak prediction, scheduled maintenance alerts, or advanced analytics.

III. GitHub:

GitHub is a code hosting platform for version control and collaboration. It lets you and others work together on projects from anywhere

GitHub essentials like repositories, branches, commits, and Pull Requests. You'll create your own Hello World repository and learn GitHub's Pull Request workflow, a popular way to create and review code.

GitHub takes things a little further than Git, offering more functionality and resources, as well as an online place to store and collaborate on projects.

A GitHub desktop app is also available, which is designed to simplify essential steps in your GitHub workflow and replace GitHub for Mac and Windows with a unified experience across both platforms.



Download link:

<https://desktop.github.com/>

Some common commands for using Git:

- git init: initializes a new Git repository.
- git clone: Creates a local copy of a project that already exists remotely.
- git merge: merges development lines together. This command is usually used to combine changes made to two distinct branches
- git pull: Updates the local development line with updates from its remote counterpart.
- git push: Updates the remote repository with any commits made locally to a branch.

Chapter 3/ Codes:

3.1 DataBase:

Firestore:

Firestore is a cloud-based platform by Google that provides powerful backend services for app development, including databases, authentication, storage, and real-time capabilities. It is widely used for mobile, web, and IoT applications.

Firestore is a **powerful backend-as-a-service (BaaS)** platform provided by **Google**, offering scalable, real-time cloud databases that are perfect for web, mobile, and IoT applications.

Here's why Firestore is an excellent choice as a database:

1. Real-time Data Synchronization

Firestore enables real-time syncing of data between the app and the cloud. This means that any changes made to the data are **immediately reflected** across all connected devices, making it ideal for live applications like chat apps, dashboards, or IoT monitoring systems.

2. Cross-platform Support

Firestore works seamlessly across **Android, iOS, and Web** platforms with easy-to-use SDKs. You can build one backend that supports multiple platforms.

3. Two Flexible Database Options

Firestore offers **two types of databases**:

- **Realtime Database:** Simple, fast, and ideal for live data syncing using a JSON tree structure.
 - **Cloud Firestore:** More structured, supports complex queries, and scalable for large apps using documents and collections.
-

4. Offline Capabilities

Both Firestore and Realtime Database support **offline access**. Apps can read and write data even when the device is offline, and sync automatically when the connection is restored.

5. Easy Integration with Other Firebase Services

Firebase works well with other tools like:

- **Firebase Authentication** (for secure login),
 - **Cloud Functions** (for server-side logic),
 - **Firebase Analytics, Firebase Hosting**, and more.
-

6. Scalable and Managed by Google

Firebase is **fully managed**, so you don't have to worry about server maintenance, scaling, or database performance. It grows with your app without the need for complex backend infrastructure.

7. Secure Access Control

With **Firebase Security Rules**, you can control who can read or write data at a very detailed level—ensuring your data is protected.

In my project, I integrated two types of Firebase databases:

- Realtime Database for live data monitoring
 - Cloud Firestore for structured data storage and analysis
-

1. Firebase Firestore

Cloud Firestore is a NoSQL cloud database provided by **Google Firebase** that allows you to store, sync, and query data for mobile and web apps in real time. It's highly scalable and flexible, making it suitable for a wide range of applications.

Key Features of Firestore:

1. Real-time Sync:

Firestore automatically syncs data between clients and the cloud in real time. Any update in the data reflects instantly across all connected devices.

2. **Flexible Data Model:**

Firestore organizes data in **collections** and **documents**, allowing for a structured but flexible format that adapts to different data needs.

3. **Offline Support:**

Firestore caches data locally so your app works seamlessly even when offline.

4. **Secure Data Access:**

Using Firebase Authentication and Firestore Security Rules, developers can control who can read or write specific data.

5. **Simple Integration:**

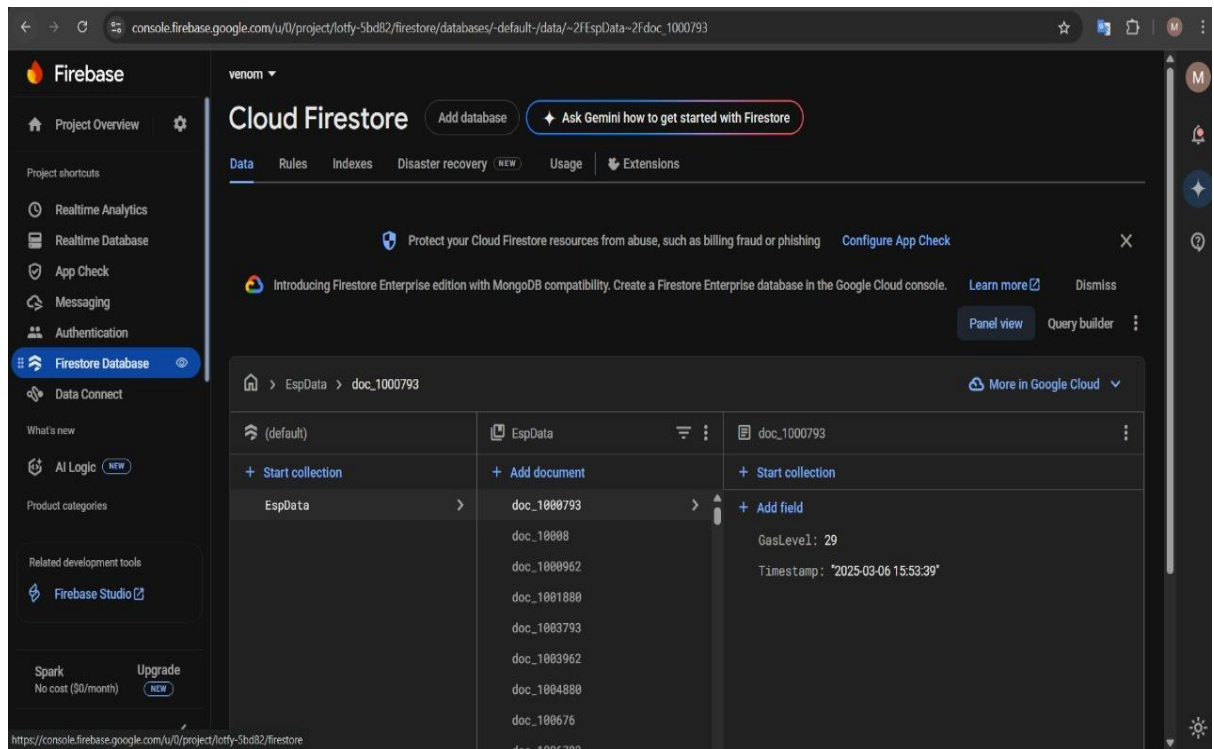
Easily integrates with Android, iOS, and Web applications with clean and simple SDKs.

Example from My Project:

In my Firebase Firestore database:

- **Collection name:** `EspData`
- **Document ID:** `doc_1000793`
- Inside this document, I store:
 - `GasLevel: 29`
 - `Timestamp: "2025-03-06 15:53:39"`

This setup is used to store gas sensor readings from a microcontroller like an **ESP32**, along with the timestamp of when the reading was taken.



2. Firebase Realtime Database

Firebase Realtime Database is a cloud-hosted NoSQL database provided by **Google Firebase**. It allows developers to store and sync data between users in **real-time**, making it perfect for apps that need instant updates such as chat applications, live dashboards, and IoT systems.

Key Features:

1. **Real-time Synchronization:**
Data is synchronized in real time across all connected clients. When one client updates the data, all other clients instantly receive the update.
2. **JSON Data Format:**
Data is stored as a single large JSON tree. It's a lightweight and simple format, making it easy to understand and use for small to medium-sized apps.

3. **Offline Support:**

Data is stored locally on the user's device while offline, and syncs automatically when the connection is restored.

4. **Cross-platform:**

Supports Android, iOS, and Web apps with built-in SDKs.

5. **Security with Rules:**

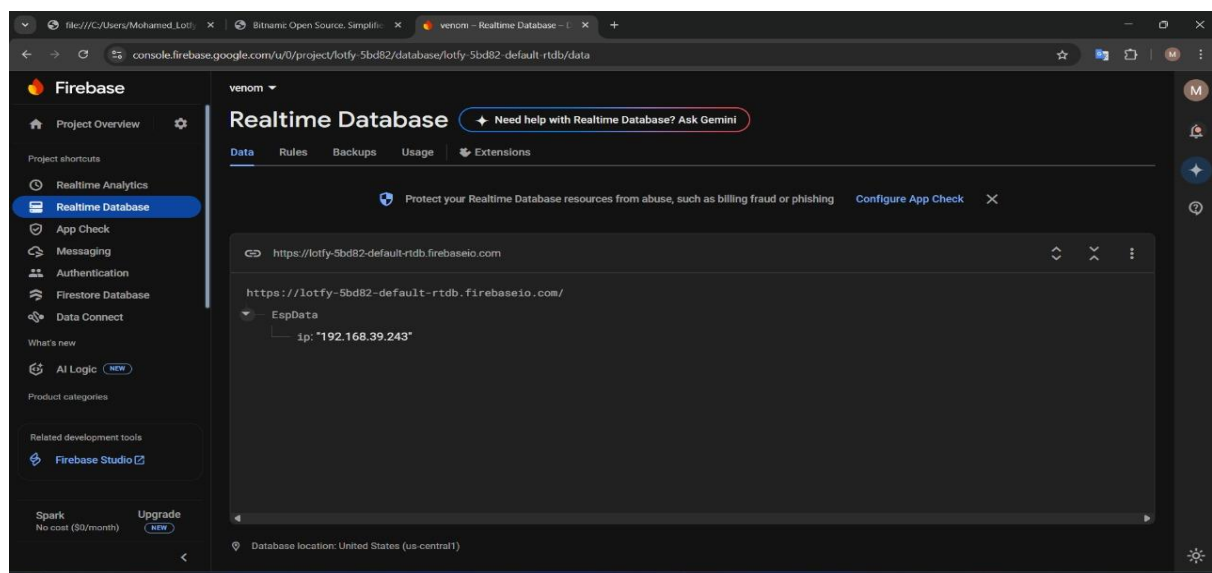
You can define custom rules to control read/write access at the node level using Firebase Realtime Database Rules.

Example from My Project:

The database is hosted at:

<https://lotfy-5bd82-default-rtdb.firebaseio.com/>

ip: 192.168.39.243



3.2 FLUTTER

Login

Login is the process where users enter their **email and password** (or other credentials) to access their personalized data or secure parts of an app.

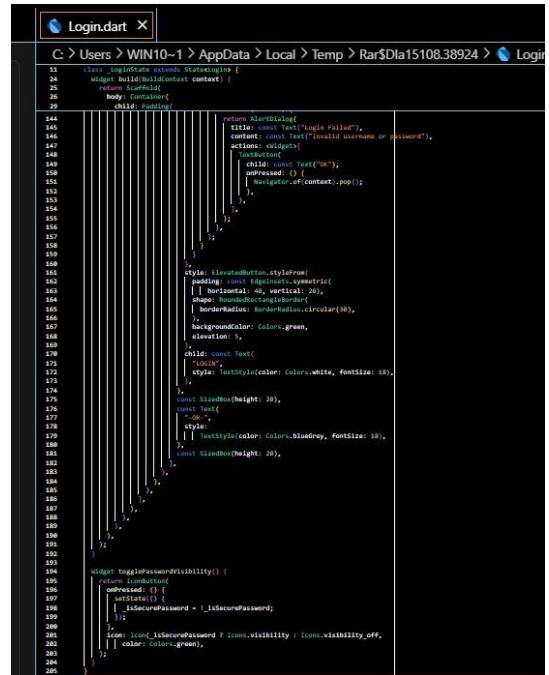
In **Flutter**, login can be implemented using different methods, and one of the most common and secure ways is by using **Firebase Authentication**.

Why Use Login in a Flutter App?

- **Security:** Keeps user data private and accessible only by them.
- **Personalization:** Allows saving user-specific settings, preferences, and history.
- **Session Management:** Helps identify users across devices or sessions.
- **Integration:** Enables linking with other services like Firestore or Realtime Database to fetch user-specific data

```
Logindart x
C:\Users\WIN10-1\AppData\Local\Temp\Rar$DIa15108.39465\> Logindart
1 import 'package:flutter/material.dart';
2 import 'package:firebase_auth/firebase_auth.dart';
3
4 class Login extends StatefulWidget {
5   const Login({super.key});
6
7   @override
8   State<StatefulWidget> createState() => _LoginState();
9 }
10
11 class _LoginState extends State<Login> {
12   bool _isSecurePassword = true;
13   final _formKey = GlobalKey<FormState>();
14
15   // Controllers
16   final TextEditingController _usernameController = TextEditingController();
17   final TextEditingController _passwordController = TextEditingController();
18
19   @override
20   Widget build(BuildContext context) {
21     return Scaffold(
22       body: Container(
23         height: 1000,
24         color: Colors.black,
25         child: Padding(
26           padding: const EdgeInsets.all(15.0),
27           child: SingleChildScrollView(
28             scrollDirection: Axis.vertical,
29             child: Column(
30               mainAxisAlignment: MainAxisAlignment.center,
31               children: [
32                 const SizedBox(height: 80),
33                 const Text(
34                   "GAS LEVEL",
35                   style: TextStyle(
36                     color: Colors.white,
37                     fontSize: 50.0,
38                     fontWeight: FontWeight.w600,
39                   ),
40                 ),
41               ],
42             ),
43           ),
44         ),
45       ),
46     );
47   }
48 }
```

```
Logindart x
C:\Users\WIN10-1\AppData\Local\Temp\Rar$DIa15108.39465\> Logindart
11 class LoginState extends State<Login> {
12   // Widgets
13   Widget build(BuildContext context) {
14     return Scaffold(
15       body: Container(
16         height: 1000,
17         color: Colors.black,
18         child: Padding(
19           padding: const EdgeInsets.all(15.0),
20           child: SingleChildScrollView(
21             scrollDirection: Axis.vertical,
22             child: Column(
23               mainAxisAlignment: MainAxisAlignment.center,
24               children: [
25                 const SizedBox(height: 80),
26                 const Text(
27                   "GAS LEVEL",
28                   style: TextStyle(
29                     color: Colors.white,
30                     fontSize: 50.0,
31                     fontWeight: FontWeight.w600,
32                   ),
33                 ),
34               ],
35             ),
36           ),
37         ),
38       ),
39     );
40   }
41 }
```



```

1  C > Users > WIN10-1 > AppData > Local > Temp > Rar$Dla15108.49669 > Start.dart
2  1 import 'package:flutter/material.dart';
3  2 import 'login.dart';
4  3 void main() {
5  4   runApp(const Start());
6  5 }
7  6 class Start extends StatelessWidget {
8  7   const Start({super.key});
9  8   @override
10  9   Widget build(BuildContext context) {
11 10     return Scaffold(
12 11       body: Container(
13 12         decoration: const BoxDecoration(
14 13           image: DecorationImage(
15 14             image: AssetImage('images/ar.jpg'),
16 15             fit: BoxFit.cover,
17 16           ),
18 17         alignment: Alignment.center,
19 18         child: Column(
20 19           mainAxisAlignment: MainAxisAlignment.center,
21 20           children: [
22 21             const Text(
23 22               'Start.com',
24 23               style: TextStyle(
25 24                 fontSize: 30.0,
26 25                 fontWeight: FontWeight.w800,
27 26                 color: Colors.white,
28 27               ),
29 28             ),
30 29             const SizedBox(height: 20),
31 30             Container(
32 31               decoration: BoxDecoration(
33 32                 boxShadow: [
34 33                   BoxShadow(
35 34                     color: Colors.green.withOpacity(0.5),
36 35                     offset: const Offset(3, 3),
37 36                     blurRadius: 20,
38 37                   ),
39 38                 ],
40 39             ),
41 40             child: ElevatedButton(
42 41               onPressed: () {
43 42                 Navigator.push(
44 43                   context,
45 44                   MaterialPageRoute(builder: (context) => const login()),
46 45                 ),
47 46             ),
48 47             style: ElevatedButton.styleFrom(
49 48               padding:
50 49                 const EdgeInsets.symmetric(horizontal: 80, vertical: 25),
51 50             ),
52 51             child: const Text(
53 52               'Start',
54 53               style: TextStyle(
55 54                 fontSize: 30,
56 55                 color: Color.fromARGB(255, 27, 201, 66),
57 56             ),
58 57           ],
59 58         ),
60 59       ),
61 60     ),
62 61   ),
63 62 ),
64 63 ),
65 64 ),
66 65 );

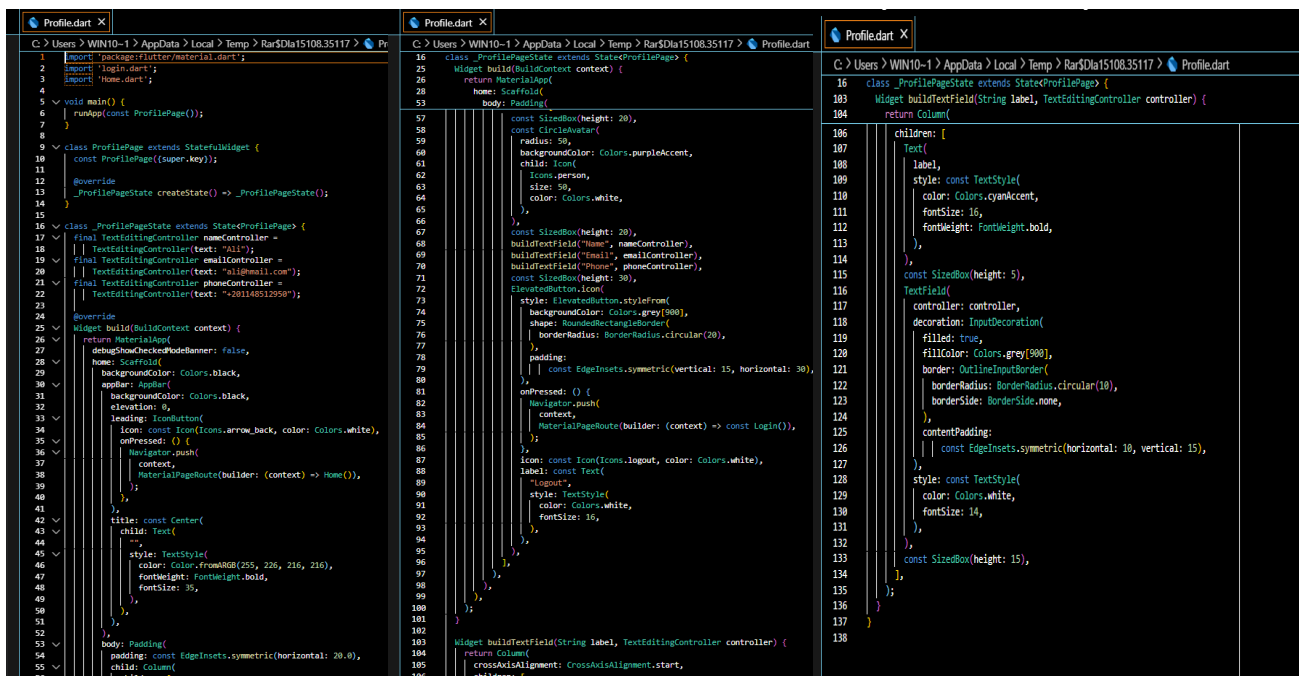
```

Profile Screen

A **User Profile** in Flutter is a screen that displays a user's personal information such as their **name, email, profile picture**, and other details. It is commonly used in apps to give users access to their account information and settings.

Why Use a Profile in Flutter Apps?

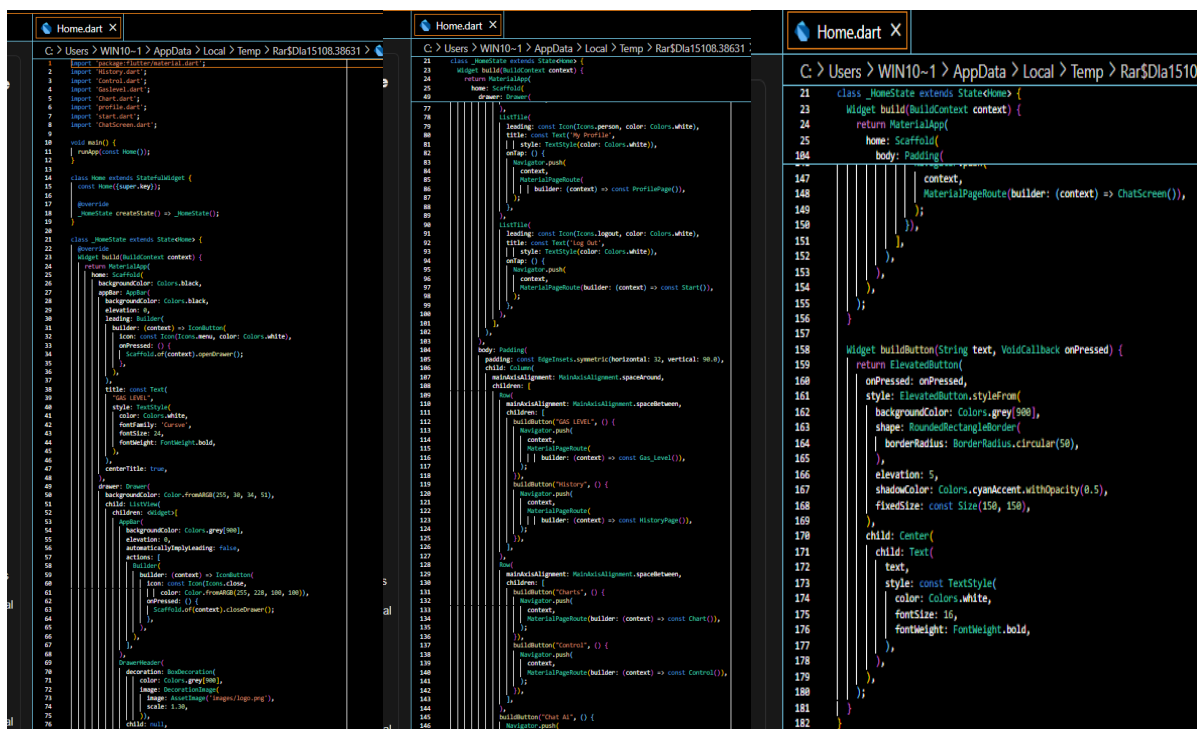
- **Personalization:** Allows users to see and manage their information.
- **User Experience:** Helps users feel more connected to the app.
- **Security & Control:** Users can update their info or sign out.
- **Integration:** Works well with backend systems like Firebase or local databases



The **Home Screen** is the first or main page the user sees after opening the app or logging in. It often displays key features, navigation buttons, or quick data (like sensor readings, user profile, etc.).

What Should Be on a Home Screen?

- App logo or name
- Welcome message (like "Welcome, User")
- Navigation to other screens (e.g., Profile, Chart, Settings)
- Important info (e.g., gas level, alerts, status)
- Easy and clean design



main.dart

The `main.dart` file is the **entry point** of every Flutter app. It contains the `main()` function, which runs the whole application

```
main.dart x
C:\Users\WIN10-1\AppData\Local\Temp\Rar$Dla15108.29765\main.dart

1 import 'package:firebase_core/firebase_core.dart';
2 import 'package:flutter/material.dart';
3 import 'Start.dart';
4
5 void main() async {
6   WidgetsFlutterBinding.ensureInitialized();
7   await Firebase.initializeApp();
8   runApp(const MyApp());
9 }
10
11 class MyApp extends StatelessWidget {
12   const MyApp({super.key});
13
14   @override
15   Widget build(BuildContext context) {
16     return const MaterialApp(
17       home: Start(),
18     );
19   }
20 }
```

Chart

A **chart** is a visual representation of data, making it easier to understand trends, comparisons, and patterns. In **Flutter**, charts are used to display dynamic or static data in a clean and interactive way.

Why Use Charts in Flutter Apps?

- **Data Visualization:** Turns raw numbers into easy-to-read visuals.
- **Trend Analysis:** Useful for showing changes over time (e.g., sensor readings).
- **Better UX:** Helps users quickly understand important information.
- **Real-Time Monitoring:** Especially useful in apps like gas sensor dashboards or IoT systems

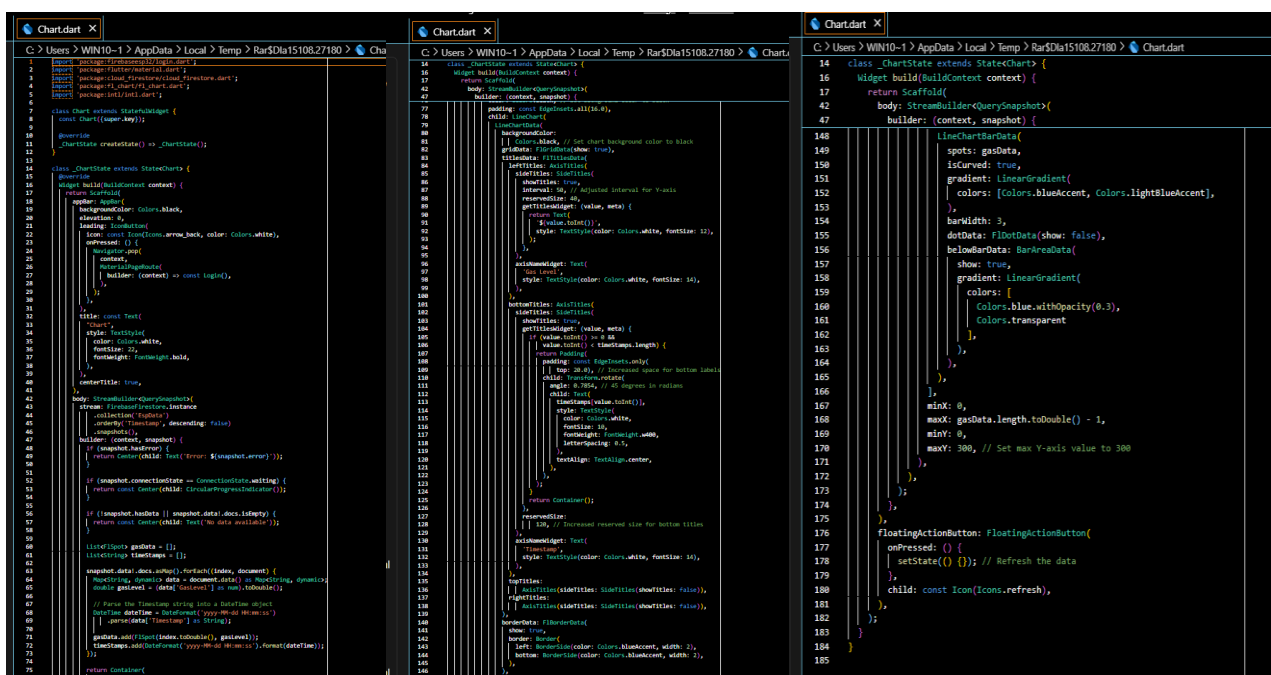
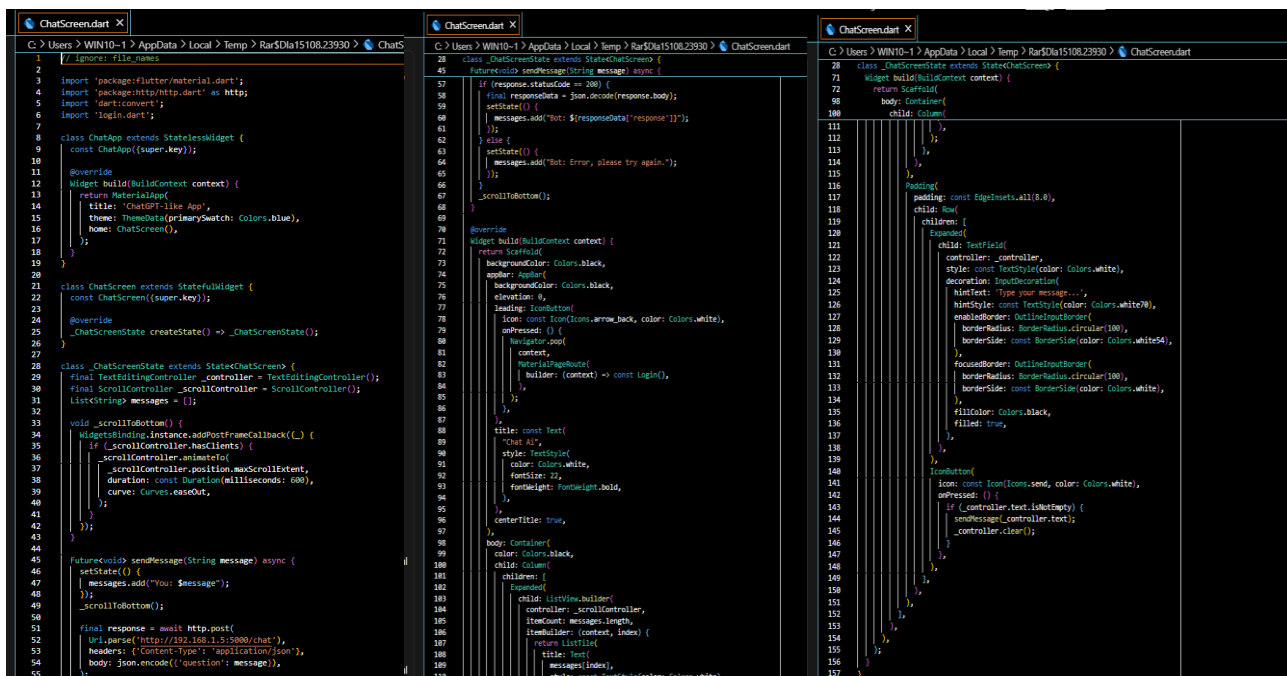


Chart Screen

A **Chart Screen** is a dedicated screen in a Flutter app where numerical or time-based data (like sensor readings) are displayed in a **graph or chart**. It allows users to understand data trends visually.

Why Use a Chart Screen?

- Shows how values change over time (e.g., gas level rising or falling).
- Helps in identifying dangerous patterns or thresholds.
- Makes technical data easier for users to understand.
- Perfect for IoT apps that collect continuous sensor readings



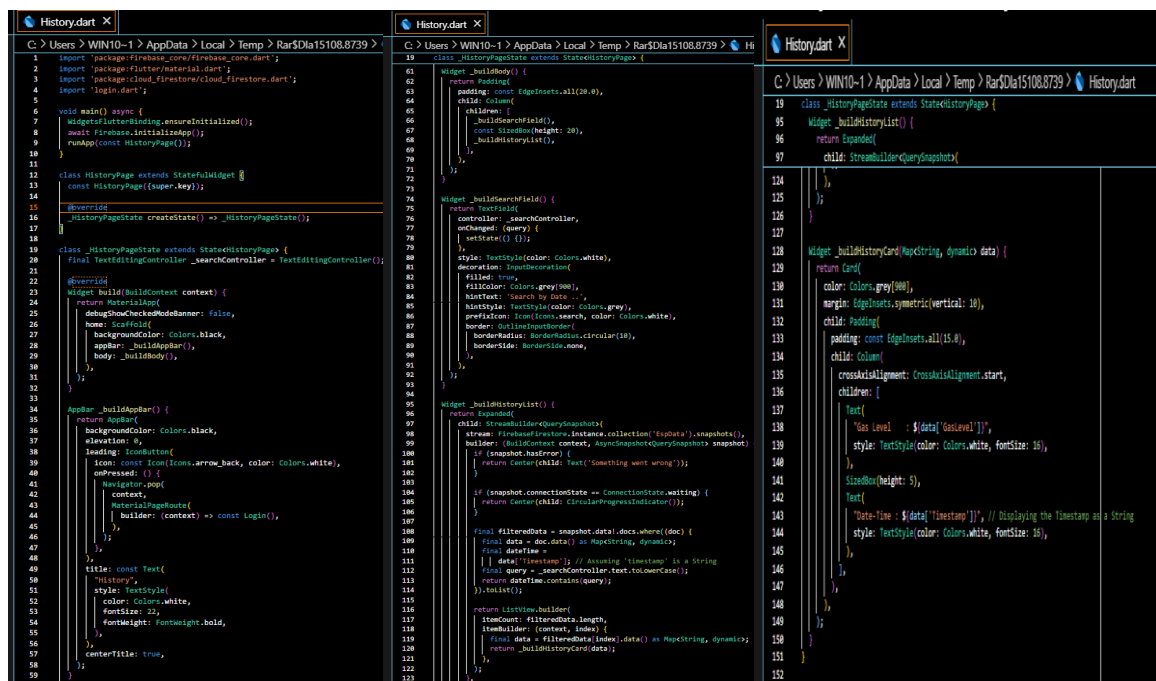
History Screen

The **History Screen** is a page in your app that displays **previous records** or **past data**, such as:

- Sensor readings (e.g., gas levels, temperature)
- Notifications or alerts
- User activities or logs

Why Use a History Screen?

- Helps users understand **what happened earlier**
- Useful for **tracking data trends over time**
- Important in safety apps (e.g., to review dangerous gas levels)
- Allows admins or users to audit events



```
History.dart
C > Users > WIN10-1 > AppData > Local > Temp > Rar$Dla15108.8739 >
1 import 'package:firebase_core/firebase_core.dart';
2 import 'package:flutter/material.dart';
3 import 'package:cloud_firestore/cloud_firestore.dart';
4 import 'login.dart';
5
6 void main() async {
7   WidgetsFlutterBinding.ensureInitialized();
8   await Firebase.initializeApp();
9   runApp(const HistoryPage());
10 }
11
12 class HistoryPage extends StatefulWidget {
13   const HistoryPage({super.key});
14
15   @override
16   _HistoryPageState createState() => _HistoryPageState();
17 }
18
19 class _HistoryPageState extends State<HistoryPage> {
20   final TextEditingController _searchController = TextEditingController();
21
22   @override
23   Widget build(BuildContext context) {
24     return MaterialApp(
25       debugShowCheckedBanner: false,
26       home: Scaffold(
27         backgroundColor: Colors.black,
28         appBar: AppBar(),
29         body: _buildBody(),
30       ),
31     );
32   }
33
34   AppBar _buildAppBar() {
35     return AppBar(
36       backgroundColor: Colors.black,
37       elevation: 0,
38       leading: IconButton(
39         icon: const Icon(Icons.arrow_back, color: Colors.white),
40         onPressed: () {
41           Navigator.pop(
42             context,
43             MaterialPageRoute(
44               builder: (context) => const Login(),
45             ),
46           );
47         },
48       ),
49       title: const Text(
50         "History",
51         style: TextStyle(
52           color: Colors.white,
53           fontSize: 22,
54           fontWeight: FontWeight.bold,
55         ),
56       ),
57       centerTitle: true,
58     );
59   }
60
61   Widget _buildBody() {
62     return Padding(
63       padding: const EdgeInsets.all(10.0),
64       child: Column(
65         children: [
66           _buildSearchField(),
67           const SizedBox(height: 20),
68           _buildHistoryList(),
69         ],
70       ),
71     );
72   }
73
74   Widget _buildSearchField() {
75     return TextField(
76       controller: _searchController,
77       onChanged: (query) {
78         _setState(() {});
79       },
80       style: TextStyle(color: Colors.white),
81       decoration: InputDecoration(
82         filled: true,
83         fillColor: Colors.grey[900],
84         hintText: "Search by Date",
85         hintStyle: TextStyle(color: Colors.grey),
86         prefixIcon: Icon(Icons.search, color: Colors.white),
87         border: OutlineInputBorder(
88           borderRadius: BorderRadius.circular(10),
89           borderSide: BorderSide.none,
90         ),
91       ),
92     );
93   }
94
95   Widget _buildHistoryList() {
96     return Expanded(
97       child: StreamBuilder<QuerySnapshot>(
98         stream: FirebaseFirestore.instance.collection('logs').snapshots(),
99         builder: (BuildContext context, AsyncSnapshot<QuerySnapshot> snapshot) {
100           if (snapshot.hasError) {
101             return Center(child: Text('Something went wrong'));
102           }
103           if (snapshot.connectionState == ConnectionState.waiting) {
104             return Center(child: CircularProgressIndicator());
105           }
106           final filteredData = snapshot.data!.docs.where((doc) {
107             final data = doc.data() as Map<String, dynamic>;
108             final dateTime =
109               [data['timestamp']] // showing timestamp is a String
110             final query = _searchController.text.toLowerCase();
111             return dateTime.contains(query);
112           }).toList();
113           return ListView.builder(
114             itemCount: filteredData.length,
115             itemBuilder: (context, index) {
116               return _buildHistoryCard(data);
117             },
118           );
119         },
120       ),
121     );
122   }
123
124   Widget _buildHistoryCard(Map<String, dynamic> data) {
125     return Card(
126       color: Colors.grey[900],
127       margin: EdgeInsets.symmetric(vertical: 10),
128       child: Padding(
129         padding: const EdgeInsets.all(15.0),
130         child: Column(
131           crossAxisAlignment: CrossAxisAlignment.start,
132           children: [
133             Text(
134               'Gas Level : ${data['GasLevel']}',
135               style: TextStyle(color: Colors.white, fontSize: 16),
136             ),
137             Text(
138               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
139               style: TextStyle(color: Colors.white, fontSize: 16),
140             ),
141           ],
142         ),
143       ),
144     );
145   }
146
147   void _setState() {}
148
149   Widget _buildHistoryCard(Map<String, dynamic> data) {
150     return Card(
151       color: Colors.grey[900],
152       margin: EdgeInsets.symmetric(vertical: 10),
153       child: Padding(
154         padding: const EdgeInsets.all(15.0),
155         child: Column(
156           crossAxisAlignment: CrossAxisAlignment.start,
157           children: [
158             Text(
159               'Gas Level : ${data['GasLevel']}',
160               style: TextStyle(color: Colors.white, fontSize: 16),
161             ),
162             Text(
163               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
164               style: TextStyle(color: Colors.white, fontSize: 16),
165             ),
166           ],
167         ),
168       ),
169     );
170   }
171
172   void _setState() {}
173
174   Widget _buildHistoryCard(Map<String, dynamic> data) {
175     return Card(
176       color: Colors.grey[900],
177       margin: EdgeInsets.symmetric(vertical: 10),
178       child: Padding(
179         padding: const EdgeInsets.all(15.0),
180         child: Column(
181           crossAxisAlignment: CrossAxisAlignment.start,
182           children: [
183             Text(
184               'Gas Level : ${data['GasLevel']}',
185               style: TextStyle(color: Colors.white, fontSize: 16),
186             ),
187             Text(
188               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
189               style: TextStyle(color: Colors.white, fontSize: 16),
190             ),
191           ],
192         ),
193       ),
194     );
195   }
196
197   void _setState() {}
198
199   Widget _buildHistoryCard(Map<String, dynamic> data) {
200     return Card(
201       color: Colors.grey[900],
202       margin: EdgeInsets.symmetric(vertical: 10),
203       child: Padding(
204         padding: const EdgeInsets.all(15.0),
205         child: Column(
206           crossAxisAlignment: CrossAxisAlignment.start,
207           children: [
208             Text(
209               'Gas Level : ${data['GasLevel']}',
210               style: TextStyle(color: Colors.white, fontSize: 16),
211             ),
212             Text(
213               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
214               style: TextStyle(color: Colors.white, fontSize: 16),
215             ),
216           ],
217         ),
218       ),
219     );
220   }
221
222   void _setState() {}
223
224   Widget _buildHistoryCard(Map<String, dynamic> data) {
225     return Card(
226       color: Colors.grey[900],
227       margin: EdgeInsets.symmetric(vertical: 10),
228       child: Padding(
229         padding: const EdgeInsets.all(15.0),
230         child: Column(
231           crossAxisAlignment: CrossAxisAlignment.start,
232           children: [
233             Text(
234               'Gas Level : ${data['GasLevel']}',
235               style: TextStyle(color: Colors.white, fontSize: 16),
236             ),
237             Text(
238               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
239               style: TextStyle(color: Colors.white, fontSize: 16),
240             ),
241           ],
242         ),
243       ),
244     );
245   }
246
247   void _setState() {}
248
249   Widget _buildHistoryCard(Map<String, dynamic> data) {
250     return Card(
251       color: Colors.grey[900],
252       margin: EdgeInsets.symmetric(vertical: 10),
253       child: Padding(
254         padding: const EdgeInsets.all(15.0),
255         child: Column(
256           crossAxisAlignment: CrossAxisAlignment.start,
257           children: [
258             Text(
259               'Gas Level : ${data['GasLevel']}',
260               style: TextStyle(color: Colors.white, fontSize: 16),
261             ),
262             Text(
263               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
264               style: TextStyle(color: Colors.white, fontSize: 16),
265             ),
266           ],
267         ),
268       ),
269     );
270   }
271
272   void _setState() {}
273
274   Widget _buildHistoryCard(Map<String, dynamic> data) {
275     return Card(
276       color: Colors.grey[900],
277       margin: EdgeInsets.symmetric(vertical: 10),
278       child: Padding(
279         padding: const EdgeInsets.all(15.0),
280         child: Column(
281           crossAxisAlignment: CrossAxisAlignment.start,
282           children: [
283             Text(
284               'Gas Level : ${data['GasLevel']}',
285               style: TextStyle(color: Colors.white, fontSize: 16),
286             ),
287             Text(
288               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
289               style: TextStyle(color: Colors.white, fontSize: 16),
290             ),
291           ],
292         ),
293       ),
294     );
295   }
296
297   void _setState() {}
298
299   Widget _buildHistoryCard(Map<String, dynamic> data) {
300     return Card(
301       color: Colors.grey[900],
302       margin: EdgeInsets.symmetric(vertical: 10),
303       child: Padding(
304         padding: const EdgeInsets.all(15.0),
305         child: Column(
306           crossAxisAlignment: CrossAxisAlignment.start,
307           children: [
308             Text(
309               'Gas Level : ${data['GasLevel']}',
310               style: TextStyle(color: Colors.white, fontSize: 16),
311             ),
312             Text(
313               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
314               style: TextStyle(color: Colors.white, fontSize: 16),
315             ),
316           ],
317         ),
318       ),
319     );
320   }
321
322   void _setState() {}
323
324   Widget _buildHistoryCard(Map<String, dynamic> data) {
325     return Card(
326       color: Colors.grey[900],
327       margin: EdgeInsets.symmetric(vertical: 10),
328       child: Padding(
329         padding: const EdgeInsets.all(15.0),
330         child: Column(
331           crossAxisAlignment: CrossAxisAlignment.start,
332           children: [
333             Text(
334               'Gas Level : ${data['GasLevel']}',
335               style: TextStyle(color: Colors.white, fontSize: 16),
336             ),
337             Text(
338               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
339               style: TextStyle(color: Colors.white, fontSize: 16),
340             ),
341           ],
342         ),
343       ),
344     );
345   }
346
347   void _setState() {}
348
349   Widget _buildHistoryCard(Map<String, dynamic> data) {
350     return Card(
351       color: Colors.grey[900],
352       margin: EdgeInsets.symmetric(vertical: 10),
353       child: Padding(
354         padding: const EdgeInsets.all(15.0),
355         child: Column(
356           crossAxisAlignment: CrossAxisAlignment.start,
357           children: [
358             Text(
359               'Gas Level : ${data['GasLevel']}',
360               style: TextStyle(color: Colors.white, fontSize: 16),
361             ),
362             Text(
363               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
364               style: TextStyle(color: Colors.white, fontSize: 16),
365             ),
366           ],
367         ),
368       ),
369     );
370   }
371
372   void _setState() {}
373
374   Widget _buildHistoryCard(Map<String, dynamic> data) {
375     return Card(
376       color: Colors.grey[900],
377       margin: EdgeInsets.symmetric(vertical: 10),
378       child: Padding(
379         padding: const EdgeInsets.all(15.0),
380         child: Column(
381           crossAxisAlignment: CrossAxisAlignment.start,
382           children: [
383             Text(
384               'Gas Level : ${data['GasLevel']}',
385               style: TextStyle(color: Colors.white, fontSize: 16),
386             ),
387             Text(
388               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
389               style: TextStyle(color: Colors.white, fontSize: 16),
390             ),
391           ],
392         ),
393       ),
394     );
395   }
396
397   void _setState() {}
398
399   Widget _buildHistoryCard(Map<String, dynamic> data) {
400     return Card(
401       color: Colors.grey[900],
402       margin: EdgeInsets.symmetric(vertical: 10),
403       child: Padding(
404         padding: const EdgeInsets.all(15.0),
405         child: Column(
406           crossAxisAlignment: CrossAxisAlignment.start,
407           children: [
408             Text(
409               'Gas Level : ${data['GasLevel']}',
410               style: TextStyle(color: Colors.white, fontSize: 16),
411             ),
412             Text(
413               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
414               style: TextStyle(color: Colors.white, fontSize: 16),
415             ),
416           ],
417         ),
418       ),
419     );
420   }
421
422   void _setState() {}
423
424   Widget _buildHistoryCard(Map<String, dynamic> data) {
425     return Card(
426       color: Colors.grey[900],
427       margin: EdgeInsets.symmetric(vertical: 10),
428       child: Padding(
429         padding: const EdgeInsets.all(15.0),
430         child: Column(
431           crossAxisAlignment: CrossAxisAlignment.start,
432           children: [
433             Text(
434               'Gas Level : ${data['GasLevel']}',
435               style: TextStyle(color: Colors.white, fontSize: 16),
436             ),
437             Text(
438               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
439               style: TextStyle(color: Colors.white, fontSize: 16),
440             ),
441           ],
442         ),
443       ),
444     );
445   }
446
447   void _setState() {}
448
449   Widget _buildHistoryCard(Map<String, dynamic> data) {
450     return Card(
451       color: Colors.grey[900],
452       margin: EdgeInsets.symmetric(vertical: 10),
453       child: Padding(
454         padding: const EdgeInsets.all(15.0),
455         child: Column(
456           crossAxisAlignment: CrossAxisAlignment.start,
457           children: [
458             Text(
459               'Gas Level : ${data['GasLevel']}',
460               style: TextStyle(color: Colors.white, fontSize: 16),
461             ),
462             Text(
463               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
464               style: TextStyle(color: Colors.white, fontSize: 16),
465             ),
466           ],
467         ),
468       ),
469     );
470   }
471
472   void _setState() {}
473
474   Widget _buildHistoryCard(Map<String, dynamic> data) {
475     return Card(
476       color: Colors.grey[900],
477       margin: EdgeInsets.symmetric(vertical: 10),
478       child: Padding(
479         padding: const EdgeInsets.all(15.0),
480         child: Column(
481           crossAxisAlignment: CrossAxisAlignment.start,
482           children: [
483             Text(
484               'Gas Level : ${data['GasLevel']}',
485               style: TextStyle(color: Colors.white, fontSize: 16),
486             ),
487             Text(
488               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
489               style: TextStyle(color: Colors.white, fontSize: 16),
490             ),
491           ],
492         ),
493       ),
494     );
495   }
496
497   void _setState() {}
498
499   Widget _buildHistoryCard(Map<String, dynamic> data) {
500     return Card(
501       color: Colors.grey[900],
502       margin: EdgeInsets.symmetric(vertical: 10),
503       child: Padding(
504         padding: const EdgeInsets.all(15.0),
505         child: Column(
506           crossAxisAlignment: CrossAxisAlignment.start,
507           children: [
508             Text(
509               'Gas Level : ${data['GasLevel']}',
510               style: TextStyle(color: Colors.white, fontSize: 16),
511             ),
512             Text(
513               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
514               style: TextStyle(color: Colors.white, fontSize: 16),
515             ),
516           ],
517         ),
518       ),
519     );
520   }
521
522   void _setState() {}
523
524   Widget _buildHistoryCard(Map<String, dynamic> data) {
525     return Card(
526       color: Colors.grey[900],
527       margin: EdgeInsets.symmetric(vertical: 10),
528       child: Padding(
529         padding: const EdgeInsets.all(15.0),
530         child: Column(
531           crossAxisAlignment: CrossAxisAlignment.start,
532           children: [
533             Text(
534               'Gas Level : ${data['GasLevel']}',
535               style: TextStyle(color: Colors.white, fontSize: 16),
536             ),
537             Text(
538               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
539               style: TextStyle(color: Colors.white, fontSize: 16),
540             ),
541           ],
542         ),
543       ),
544     );
545   }
546
547   void _setState() {}
548
549   Widget _buildHistoryCard(Map<String, dynamic> data) {
550     return Card(
551       color: Colors.grey[900],
552       margin: EdgeInsets.symmetric(vertical: 10),
553       child: Padding(
554         padding: const EdgeInsets.all(15.0),
555         child: Column(
556           crossAxisAlignment: CrossAxisAlignment.start,
557           children: [
558             Text(
559               'Gas Level : ${data['GasLevel']}',
560               style: TextStyle(color: Colors.white, fontSize: 16),
561             ),
562             Text(
563               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
564               style: TextStyle(color: Colors.white, fontSize: 16),
565             ),
566           ],
567         ),
568       ),
569     );
570   }
571
572   void _setState() {}
573
574   Widget _buildHistoryCard(Map<String, dynamic> data) {
575     return Card(
576       color: Colors.grey[900],
577       margin: EdgeInsets.symmetric(vertical: 10),
578       child: Padding(
579         padding: const EdgeInsets.all(15.0),
580         child: Column(
581           crossAxisAlignment: CrossAxisAlignment.start,
582           children: [
583             Text(
584               'Gas Level : ${data['GasLevel']}',
585               style: TextStyle(color: Colors.white, fontSize: 16),
586             ),
587             Text(
588               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
589               style: TextStyle(color: Colors.white, fontSize: 16),
590             ),
591           ],
592         ),
593       ),
594     );
595   }
596
597   void _setState() {}
598
599   Widget _buildHistoryCard(Map<String, dynamic> data) {
600     return Card(
601       color: Colors.grey[900],
602       margin: EdgeInsets.symmetric(vertical: 10),
603       child: Padding(
604         padding: const EdgeInsets.all(15.0),
605         child: Column(
606           crossAxisAlignment: CrossAxisAlignment.start,
607           children: [
608             Text(
609               'Gas Level : ${data['GasLevel']}',
610               style: TextStyle(color: Colors.white, fontSize: 16),
611             ),
612             Text(
613               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
614               style: TextStyle(color: Colors.white, fontSize: 16),
615             ),
616           ],
617         ),
618       ),
619     );
620   }
621
622   void _setState() {}
623
624   Widget _buildHistoryCard(Map<String, dynamic> data) {
625     return Card(
626       color: Colors.grey[900],
627       margin: EdgeInsets.symmetric(vertical: 10),
628       child: Padding(
629         padding: const EdgeInsets.all(15.0),
630         child: Column(
631           crossAxisAlignment: CrossAxisAlignment.start,
632           children: [
633             Text(
634               'Gas Level : ${data['GasLevel']}',
635               style: TextStyle(color: Colors.white, fontSize: 16),
636             ),
637             Text(
638               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
639               style: TextStyle(color: Colors.white, fontSize: 16),
640             ),
641           ],
642         ),
643       ),
644     );
645   }
646
647   void _setState() {}
648
649   Widget _buildHistoryCard(Map<String, dynamic> data) {
650     return Card(
651       color: Colors.grey[900],
652       margin: EdgeInsets.symmetric(vertical: 10),
653       child: Padding(
654         padding: const EdgeInsets.all(15.0),
655         child: Column(
656           crossAxisAlignment: CrossAxisAlignment.start,
657           children: [
658             Text(
659               'Gas Level : ${data['GasLevel']}',
660               style: TextStyle(color: Colors.white, fontSize: 16),
661             ),
662             Text(
663               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
664               style: TextStyle(color: Colors.white, fontSize: 16),
665             ),
666           ],
667         ),
668       ),
669     );
670   }
671
672   void _setState() {}
673
674   Widget _buildHistoryCard(Map<String, dynamic> data) {
675     return Card(
676       color: Colors.grey[900],
677       margin: EdgeInsets.symmetric(vertical: 10),
678       child: Padding(
679         padding: const EdgeInsets.all(15.0),
680         child: Column(
681           crossAxisAlignment: CrossAxisAlignment.start,
682           children: [
683             Text(
684               'Gas Level : ${data['GasLevel']}',
685               style: TextStyle(color: Colors.white, fontSize: 16),
686             ),
687             Text(
688               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
689               style: TextStyle(color: Colors.white, fontSize: 16),
690             ),
691           ],
692         ),
693       ),
694     );
695   }
696
697   void _setState() {}
698
699   Widget _buildHistoryCard(Map<String, dynamic> data) {
700     return Card(
701       color: Colors.grey[900],
702       margin: EdgeInsets.symmetric(vertical: 10),
703       child: Padding(
704         padding: const EdgeInsets.all(15.0),
705         child: Column(
706           crossAxisAlignment: CrossAxisAlignment.start,
707           children: [
708             Text(
709               'Gas Level : ${data['GasLevel']}',
710               style: TextStyle(color: Colors.white, fontSize: 16),
711             ),
712             Text(
713               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
714               style: TextStyle(color: Colors.white, fontSize: 16),
715             ),
716           ],
717         ),
718       ),
719     );
720   }
721
722   void _setState() {}
723
724   Widget _buildHistoryCard(Map<String, dynamic> data) {
725     return Card(
726       color: Colors.grey[900],
727       margin: EdgeInsets.symmetric(vertical: 10),
728       child: Padding(
729         padding: const EdgeInsets.all(15.0),
730         child: Column(
731           crossAxisAlignment: CrossAxisAlignment.start,
732           children: [
733             Text(
734               'Gas Level : ${data['GasLevel']}',
735               style: TextStyle(color: Colors.white, fontSize: 16),
736             ),
737             Text(
738               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
739               style: TextStyle(color: Colors.white, fontSize: 16),
740             ),
741           ],
742         ),
743       ),
744     );
745   }
746
747   void _setState() {}
748
749   Widget _buildHistoryCard(Map<String, dynamic> data) {
750     return Card(
751       color: Colors.grey[900],
752       margin: EdgeInsets.symmetric(vertical: 10),
753       child: Padding(
754         padding: const EdgeInsets.all(15.0),
755         child: Column(
756           crossAxisAlignment: CrossAxisAlignment.start,
757           children: [
758             Text(
759               'Gas Level : ${data['GasLevel']}',
760               style: TextStyle(color: Colors.white, fontSize: 16),
761             ),
762             Text(
763               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
764               style: TextStyle(color: Colors.white, fontSize: 16),
765             ),
766           ],
767         ),
768       ),
769     );
770   }
771
772   void _setState() {}
773
774   Widget _buildHistoryCard(Map<String, dynamic> data) {
775     return Card(
776       color: Colors.grey[900],
777       margin: EdgeInsets.symmetric(vertical: 10),
778       child: Padding(
779         padding: const EdgeInsets.all(15.0),
780         child: Column(
781           crossAxisAlignment: CrossAxisAlignment.start,
782           children: [
783             Text(
784               'Gas Level : ${data['GasLevel']}',
785               style: TextStyle(color: Colors.white, fontSize: 16),
786             ),
787             Text(
788               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
789               style: TextStyle(color: Colors.white, fontSize: 16),
790             ),
791           ],
792         ),
793       ),
794     );
795   }
796
797   void _setState() {}
798
799   Widget _buildHistoryCard(Map<String, dynamic> data) {
800     return Card(
801       color: Colors.grey[900],
802       margin: EdgeInsets.symmetric(vertical: 10),
803       child: Padding(
804         padding: const EdgeInsets.all(15.0),
805         child: Column(
806           crossAxisAlignment: CrossAxisAlignment.start,
807           children: [
808             Text(
809               'Gas Level : ${data['GasLevel']}',
810               style: TextStyle(color: Colors.white, fontSize: 16),
811             ),
812             Text(
813               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
814               style: TextStyle(color: Colors.white, fontSize: 16),
815             ),
816           ],
817         ),
818       ),
819     );
820   }
821
822   void _setState() {}
823
824   Widget _buildHistoryCard(Map<String, dynamic> data) {
825     return Card(
826       color: Colors.grey[900],
827       margin: EdgeInsets.symmetric(vertical: 10),
828       child: Padding(
829         padding: const EdgeInsets.all(15.0),
830         child: Column(
831           crossAxisAlignment: CrossAxisAlignment.start,
832           children: [
833             Text(
834               'Gas Level : ${data['GasLevel']}',
835               style: TextStyle(color: Colors.white, fontSize: 16),
836             ),
837             Text(
838               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
839               style: TextStyle(color: Colors.white, fontSize: 16),
840             ),
841           ],
842         ),
843       ),
844     );
845   }
846
847   void _setState() {}
848
849   Widget _buildHistoryCard(Map<String, dynamic> data) {
850     return Card(
851       color: Colors.grey[900],
852       margin: EdgeInsets.symmetric(vertical: 10),
853       child: Padding(
854         padding: const EdgeInsets.all(15.0),
855         child: Column(
856           crossAxisAlignment: CrossAxisAlignment.start,
857           children: [
858             Text(
859               'Gas Level : ${data['GasLevel']}',
860               style: TextStyle(color: Colors.white, fontSize: 16),
861             ),
862             Text(
863               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
864               style: TextStyle(color: Colors.white, fontSize: 16),
865             ),
866           ],
867         ),
868       ),
869     );
870   }
871
872   void _setState() {}
873
874   Widget _buildHistoryCard(Map<String, dynamic> data) {
875     return Card(
876       color: Colors.grey[900],
877       margin: EdgeInsets.symmetric(vertical: 10),
878       child: Padding(
879         padding: const EdgeInsets.all(15.0),
880         child: Column(
881           crossAxisAlignment: CrossAxisAlignment.start,
882           children: [
883             Text(
884               'Gas Level : ${data['GasLevel']}',
885               style: TextStyle(color: Colors.white, fontSize: 16),
886             ),
887             Text(
888               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
889               style: TextStyle(color: Colors.white, fontSize: 16),
890             ),
891           ],
892         ),
893       ),
894     );
895   }
896
897   void _setState() {}
898
899   Widget _buildHistoryCard(Map<String, dynamic> data) {
900     return Card(
901       color: Colors.grey[900],
902       margin: EdgeInsets.symmetric(vertical: 10),
903       child: Padding(
904         padding: const EdgeInsets.all(15.0),
905         child: Column(
906           crossAxisAlignment: CrossAxisAlignment.start,
907           children: [
908             Text(
909               'Gas Level : ${data['GasLevel']}',
910               style: TextStyle(color: Colors.white, fontSize: 16),
911             ),
912             Text(
913               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
914               style: TextStyle(color: Colors.white, fontSize: 16),
915             ),
916           ],
917         ),
918       ),
919     );
920   }
921
922   void _setState() {}
923
924   Widget _buildHistoryCard(Map<String, dynamic> data) {
925     return Card(
926       color: Colors.grey[900],
927       margin: EdgeInsets.symmetric(vertical: 10),
928       child: Padding(
929         padding: const EdgeInsets.all(15.0),
930         child: Column(
931           crossAxisAlignment: CrossAxisAlignment.start,
932           children: [
933             Text(
934               'Gas Level : ${data['GasLevel']}',
935               style: TextStyle(color: Colors.white, fontSize: 16),
936             ),
937             Text(
938               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
939               style: TextStyle(color: Colors.white, fontSize: 16),
940             ),
941           ],
942         ),
943       ),
944     );
945   }
946
947   void _setState() {}
948
949   Widget _buildHistoryCard(Map<String, dynamic> data) {
950     return Card(
951       color: Colors.grey[900],
952       margin: EdgeInsets.symmetric(vertical: 10),
953       child: Padding(
954         padding: const EdgeInsets.all(15.0),
955         child: Column(
956           crossAxisAlignment: CrossAxisAlignment.start,
957           children: [
958             Text(
959               'Gas Level : ${data['GasLevel']}',
960               style: TextStyle(color: Colors.white, fontSize: 16),
961             ),
962             Text(
963               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
964               style: TextStyle(color: Colors.white, fontSize: 16),
965             ),
966           ],
967         ),
968       ),
969     );
970   }
971
972   void _setState() {}
973
974   Widget _buildHistoryCard(Map<String, dynamic> data) {
975     return Card(
976       color: Colors.grey[900],
977       margin: EdgeInsets.symmetric(vertical: 10),
978       child: Padding(
979         padding: const EdgeInsets.all(15.0),
980         child: Column(
981           crossAxisAlignment: CrossAxisAlignment.start,
982           children: [
983             Text(
984               'Gas Level : ${data['GasLevel']}',
985               style: TextStyle(color: Colors.white, fontSize: 16),
986             ),
987             Text(
988               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
989               style: TextStyle(color: Colors.white, fontSize: 16),
990             ),
991           ],
992         ),
993       ),
994     );
995   }
996
997   void _setState() {}
998
999   Widget _buildHistoryCard(Map<String, dynamic> data) {
1000     return Card(
1001       color: Colors.grey[900],
1002       margin: EdgeInsets.symmetric(vertical: 10),
1003       child: Padding(
1004         padding: const EdgeInsets.all(15.0),
1005         child: Column(
1006           crossAxisAlignment: CrossAxisAlignment.start,
1007           children: [
1008             Text(
1009               'Gas Level : ${data['GasLevel']}',
1010               style: TextStyle(color: Colors.white, fontSize: 16),
1011             ),
1012             Text(
1013               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
1014               style: TextStyle(color: Colors.white, fontSize: 16),
1015             ),
1016           ],
1017         ),
1018       ),
1019     );
1020   }
1021
1022   void _setState() {}
1023
1024   Widget _buildHistoryCard(Map<String, dynamic> data) {
1025     return Card(
1026       color: Colors.grey[900],
1027       margin: EdgeInsets.symmetric(vertical: 10),
1028       child: Padding(
1029         padding: const EdgeInsets.all(15.0),
1030         child: Column(
1031           crossAxisAlignment: CrossAxisAlignment.start,
1032           children: [
1033             Text(
1034               'Gas Level : ${data['GasLevel']}',
1035               style: TextStyle(color: Colors.white, fontSize: 16),
1036             ),
1037             Text(
1038               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
1039               style: TextStyle(color: Colors.white, fontSize: 16),
1040             ),
1041           ],
1042         ),
1043       ),
1044     );
1045   }
1046
1047   void _setState() {}
1048
1049   Widget _buildHistoryCard(Map<String, dynamic> data) {
1050     return Card(
1051       color: Colors.grey[900],
1052       margin: EdgeInsets.symmetric(vertical: 10),
1053       child: Padding(
1054         padding: const EdgeInsets.all(15.0),
1055         child: Column(
1056           crossAxisAlignment: CrossAxisAlignment.start,
1057           children: [
1058             Text(
1059               'Gas Level : ${data['GasLevel']}',
1060               style: TextStyle(color: Colors.white, fontSize: 16),
1061             ),
1062             Text(
1063               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
1064               style: TextStyle(color: Colors.white, fontSize: 16),
1065             ),
1066           ],
1067         ),
1068       ),
1069     );
1070   }
1071
1072   void _setState() {}
1073
1074   Widget _buildHistoryCard(Map<String, dynamic> data) {
1075     return Card(
1076       color: Colors.grey[900],
1077       margin: EdgeInsets.symmetric(vertical: 10),
1078       child: Padding(
1079         padding: const EdgeInsets.all(15.0),
1080         child: Column(
1081           crossAxisAlignment: CrossAxisAlignment.start,
1082           children: [
1083             Text(
1084               'Gas Level : ${data['GasLevel']}',
1085               style: TextStyle(color: Colors.white, fontSize: 16),
1086             ),
1087             Text(
1088               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
1089               style: TextStyle(color: Colors.white, fontSize: 16),
1090             ),
1091           ],
1092         ),
1093       ),
1094     );
1095   }
1096
1097   void _setState() {}
1098
1099   Widget _buildHistoryCard(Map<String, dynamic> data) {
1100     return Card(
1101       color: Colors.grey[900],
1102       margin: EdgeInsets.symmetric(vertical: 10),
1103       child: Padding(
1104         padding: const EdgeInsets.all(15.0),
1105         child: Column(
1106           crossAxisAlignment: CrossAxisAlignment.start,
1107           children: [
1108             Text(
1109               'Gas Level : ${data['GasLevel']}',
1110               style: TextStyle(color: Colors.white, fontSize: 16),
1111             ),
1112             Text(
1113               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
1114               style: TextStyle(color: Colors.white, fontSize: 16),
1115             ),
1116           ],
1117         ),
1118       ),
1119     );
1120   }
1121
1122   void _setState() {}
1123
1124   Widget _buildHistoryCard(Map<String, dynamic> data) {
1125     return Card(
1126       color: Colors.grey[900],
1127       margin: EdgeInsets.symmetric(vertical: 10),
1128       child: Padding(
1129         padding: const EdgeInsets.all(15.0),
1130         child: Column(
1131           crossAxisAlignment: CrossAxisAlignment.start,
1132           children: [
1133             Text(
1134               'Gas Level : ${data['GasLevel']}',
1135               style: TextStyle(color: Colors.white, fontSize: 16),
1136             ),
1137             Text(
1138               'Date-Time : ${data['timestamp']}', // Displaying the timestamp as a String
1139               style: TextStyle(color: Colors.white, fontSize: 16),
1140             ),
1141           ],
1142         ),
1143       ),
1144     );
1145   }
1146
1147   void _setState() {}
1148
1149  
```

Email

```
Email.dart X
C:\Users\WIN10-1\AppData\Local\Temp\Rar$Dla15108.23476\Email.dar
1 import 'package:firebaseesp32/Home.dart';
2 import 'package:firebaseesp32/Notificationess.dart';
3 import 'package:flutter/material.dart';
4 import 'package:cloud_firestore/cloud_firestore.dart';
5
6 import 'package:http/http.dart' as http;
7
8 void main() {
9   runApp(const MaterialApp(
10     home: Emails(),
11     debugShowCheckedModeBanner: false,
12   ));
13 }
14
15 class Emails extends StatefulWidget {
16   const Emails({Key? key}) : super(key: key);
17
18   @override
19   _EmailsState createState() => _EmailsState();
20 }
21
22 class _EmailsState extends State<Emails> {
23   DateTime? _lastEmailSentTime;
24
25   @override
26   void initState() {
27     super.initState();
28     _listenForGasLevelUpdates();
29
30     WidgetsBinding.instance.addPostFrameCallback((_) {
31       Navigator.push(
32         context,
33         MaterialPageRoute(builder: (context) => const notifications()),
34       ).then((_) {
35         Navigator.push(
36           context,
37           MaterialPageRoute(builder: (context) => const Home()),
38         );
39       });
40     });
41 }
42
43
44 Email.dart X
C:\Users\WIN10-1\AppData\Local\Temp\Rar$Dla15108.23476\Email.dar
22 class _EmailsState extends State<Emails> {
43   void _listenForGasLevelUpdates() {
44     FirebaseFirestore.instance
45       .collection('EspData')
46       .orderBy('timestamp', descending: true)
47       .limit(1)
48       .snapshots()
49       .listen((snapshot) async {
50         if (snapshot.docs.isNotEmpty) {
51           Map<String, dynamic> data = snapshot.docs.first.data();
52           int gasLevel = (data['gasLevel'] as num).toInt();
53
54           if (gasLevel > 50) {
55             DateTime now = DateTime.now();
56             if (_lastEmailSentTime == null ||
57               now.difference(_lastEmailSentTime!) >
58               const Duration(seconds: 60)) {
59               _lastEmailSentTime = now;
60               await _sendEmailNotification(gasLevel);
61             }
62           }
63         }
64       });
65   }
66
67   Future<void> _sendEmailNotification(int gasLevel) async {
68     final response = await http.post(
69       Uri.parse('http://192.168.1.3/email/alert.php'),
70       body: {'gas_level': gasLevel.toString()},
71     );
72
73     if (response.statusCode == 200) {
74       print("Send Success");
75     } else {
76       print("Error : ${response.statusCode}");
77     }
78   }
79
80   @override
81   Widget build(BuildContext context) {
82     return Scaffold(
83       body: Container(
84         color: Colors.black,
85       ),
86     );
87   }
88 }
```

Control

<pre> 1 import 'package:flutter/material.dart'; 2 import 'package:web_socket_channel/web_socket_channel.dart'; 3 import 'package:firebase_database/firebase_database.dart'; 4 import 'package:flutter/services.dart'; // For Clipboard 5 import 'home.dart'; 6 7 void main() { 8 runApp(const Control()); 9 } 10 11 class Control extends StatelessWidget { 12 const Control({super.key}); 13 14 @override 15 Widget build(BuildContext context) { 16 return const MaterialApp(17 home: App(), 18); 19 } 20 } 21 22 class App extends StatelessWidget { 23 const App({super.key}); 24 25 @override 26 State<App> createState() => _AppState(); 27 } 28 29 class _AppState extends State<App> { 30 bool switch1 = true; 31 32 final TextEditingController ipController = TextEditingController(); 33 WebSocketChannel? channel; 34 bool isConnected = false; 35 String receivedMessage = ''; 36 String firebaseIp = ''; 37 38 final DatabaseReference _database = FirebaseDatabase.instance.ref(); 39 40 @override 41 void initState() { 42 super.initState(); 43 fetchIpFromFirebase(); // Fetch the IP address on app startup 44 } 45 46 // Fetch IP from Firebase Realtime Database 47 Future<void> fetchIpFromFirebase() async { 48 DatabaseReference reference = _database.child('EspData/ip'); 49 reference.once().then((DatabaseEvent event) { 50 setState(() { 51 firebaseIp = event.snapshot.value.toString(); 52 ipController.text = '\$firebaseIp:81'; // Append :81 to the IP address 53 }); 54 }).catchError((error) { 55 print('Error fetching IP: \$error'); 56 }); 57 } 58 </pre>	<pre> 29 class _AppState extends State<App> { 30 // Connect to WebSocket 31 Future<void> connect() async { 32 if (ipController.text.isNotEmpty) { 33 final url = 'ws://\$ipController.text'; 34 try { 35 channel = WebSocketChannel.connect(Uri.parse(url)); 36 setState(() { 37 isConnected = true; 38 }); 39 channel!.stream.listen((message) { 40 setState(() { 41 receivedMessage = message; 42 }); 43 print('Received: \$message'); 44 }, onError: (error) { 45 print('Error: \$error'); 46 setState(() { 47 isConnected = false; 48 }); 49 }, onDone: () { 50 setState(() { 51 isConnected = false; 52 }); 53 }); 54 } catch (e) { 55 print('Connection failed: \$e'); 56 setState(() { 57 isConnected = false; 58 }); 59 } 60 } else { 61 print('Please enter a valid IP address.');</pre>
---	---

<pre> 111 @override 112 Widget build(BuildContext context) { 113 return Scaffold(114 appBar: AppBar(115 backgroundColor: Colors.black, 116 elevation: 4, 117 leading: IconButton(118 icon: const Icon(Icons.arrow_back, color: Colors.white), 119 onPressed: () { 120 Navigator.push(121 context, 122 MaterialPageRoute(builder: (context) => Home()), 123); 124 }, 125), 126 title: const Center(127 child: Text(128 "On", 129 style: TextStyle(130 color: Color.fromARGB(255, 226, 216, 216), 131 fontWeight: FontWeight.bold, 132 fontSize: 35, 133), 134), 135), 136), 137 body: Container(138 color: Colors.black, 139 child: Column(140 children: [141 Row(142 mainAxisAlignment: MainAxisAlignment.spaceEvenly, 143 children: [144 SizedBox(145 width: 150, 146 child: TextField(147 style: const TextStyle(color: Colors.white), 148 controller: ipController, 149 decoration: const InputDecoration(150 border: OutlineInputBorder(), 151 labelText: 'IP', 152), 153), 154 ElevatedButton(155 onPressed: connect, 156 child: const Text('Connect'), 157), 158 IconButton(159 icon: const Icon(Icons.copy), 160 onPressed: copyToClipboard, // Use the copy function 161), 162], 163), 164], 165), </pre>	<pre> 166 if (isConnected) 167 const Text("Connected", style: TextStyle(color: Colors.green)) 168 else 169 const Text("Not connected", style: TextStyle(color: Colors.red)), 170 const SizedBox(height: 20), 171 Text("Received: \$receivedMessage", 172 Row(173 mainAxisAlignment: MainAxisAlignment.spaceEvenly, 174 children: [175 const Text(176 "Header:", 177 style: TextStyle(178 color: Colors.white, 179), 180), 181 Switch(182 value: switch1, 183 onChanged: (value) { 184 setState(() { 185 switch1 = value; 186 channel!.sink.add('LED1:\$(value ? 1 : 0)'); 187 }); 188 }, 189), 190], 191), 192), 193], 194), 195], 196), 197 } </pre>
---	--

Notifications

Notifications are messages shown to the user **outside the app UI** — either in the system tray (push notification) or as pop-ups (local notification)

```
Notificationioess.dart X
C:\Users\WIN10-1\AppData\Local\Temp\Rar$Dla15108.21079> Notificationioess.dart
1 import 'package:flutter/material.dart';
2 import 'package:cloud_firestore/cloud_firestore.dart';
3 import 'package:flutter_local_notifications/flutter_local_notifications.dart';
4
5 class notifications extends StatefulWidget {
6   const notifications({super.key});
7
8   @override
9   _notificationsState createState() => _notificationsState();
10 }
11
12 class _notificationsState extends State<notifications> {
13   final FlutterLocalNotificationsPlugin flutterLocalNotificationsPlugin =
14     FlutterLocalNotificationsPlugin();
15
16   @override
17   void initState() {
18     super.initState();
19     _initializeNotifications();
20     _listenForGasLevelUpdates();
21     Future.delayed(const Duration(seconds: 0), () {
22       Navigator.pop(context);
23     });
24   }
25
26   // Initialize local notifications
27   Future<void> _initializeNotifications() async {
28     const AndroidInitializationSettings initializationSettingsAndroid =
29       AndroidInitializationSettings('@mipmap/ic_launcher');
30     const InitializationSettings initializationSettings =
31       InitializationSettings(android: initializationSettingsAndroid);
32     await flutterLocalNotificationsPlugin.initialize(initializationSettings);
33   }
34
35   Future<void> _showNotification(int gasLevel) async {
36     const AndroidNotificationDetails androidPlatformChannelSpecifics =
37       AndroidNotificationDetails(
38         'gas_level_channel',
39         'Gas Level Alerts',
40         importance: Importance.high,
41         priority: Priority.high,
42       );
43     const NotificationDetails platformChannelSpecifics =
44       NotificationDetails(android: androidPlatformChannelSpecifics);
45 }
```

```
Notificationioess.dart X
C:\Users\WIN10-1\AppData\Local\Temp\Rar$Dla15108.21079> Notificationioess.dart
12 class _notificationsState extends State<notifications> {
35   Future<void> _showNotification(int gasLevel) async {
43     const NotificationDetails platformChannelSpecifics =
45       NotificationDetails(
46         android: androidPlatformChannelSpecifics,
47         ios: null,
48         macOS: null,
49         'Gas Level has reached $gasLevel ppm!',
50         platformChannelSpecifics,
51       );
52   }
53
54   void _listenForGasLevelUpdates() {
55     FirebaseFirestore.instance
56       .collection('EspData')
57       .orderBy('Timestamp', descending: true)
58       .limit(1)
59       .snapshots()
60       .listen((snapshot) {
61         if (snapshot.docs.isNotEmpty) {
62           Map<String, dynamic> data = snapshot.docs.first.data();
63           int gasLevel = (data['GasLevel'] as num).toInt();
64
65           if (gasLevel > 50) {
66             _showNotification(gasLevel);
67           }
68         }
69       });
70   }
71
72   @override
73   Widget build(BuildContext context) {
74     return Scaffold(
75       body: Container(
76         color: Colors.black,
77       ),
78     );
79   }
80 }
```

Gas Level

Gas Level refers to the concentration of a specific gas (like methane, LPG, etc.) detected by a **gas sensor** (e.g., MQ-2) connected to a microcontroller (like ESP32 or Arduino). In Flutter, we can display and track this gas level in real time by connecting to a database such as **Firebase**.

Why Display Gas Level in Flutter?

- **Live Monitoring:** Track gas presence or leaks as they happen.
- **Data Visualization:** Show gas readings over time with charts or graphs.
- **User Alerts:** Send warnings if the gas level exceeds a safe threshold.
- **IoT Integration:** Combine sensor hardware with a smart app interface.

```
Gaslevel.dart
C:\Users\WIN10-1> AppData\Local\Temp\Rar$Dla15108.33634 > Gaslevel.dart
1 import 'package:flutter/material.dart';
2 import 'package:cloud_firestore/cloud_firestore.dart';
3 import 'package:animate_do/animate_do.dart';
4 import 'package:flutter_local_notifications/flutter_local_notifications.dart';
5
6 class Gas_Level extends StatefulWidget {
7   const Gas_Level({super.key});
8
9   @override
10  _Gas_LevelState createState() => _Gas_LevelState();
11
12
13  class _Gas_LevelState extends State<Gas_Level> {
14    final FlutterLocalNotificationsPlugin flutterLocalNotificationsPlugin =
15      FlutterLocalNotificationsPlugin();
16
17    @override
18    void initState() {
19      super.initState();
20      initializeNotifications();
21      _listenForGasLevelUpdates();
22    }
23
24    Future<void> _initializeNotifications() async {
25      const AndroidInitializationSettings initializationSettingsAndroid =
26        AndroidInitializationSettings('mipmap/ic_launcher');
27      const InitializationSettings initializationSettings =
28        InitializationSettings(android: initializationSettingsAndroid);
29      await flutterLocalNotificationsPlugin.initialize(initializationSettings);
30    }
31
32    Future<void> _showNotification(int gaslevel) async {
33      const AndroidNotificationDetails androidPlatformChannelSpecifics =
34        AndroidNotificationDetails(
35          'gas_level_channel',
36          'Gas Level Alerts',
37          importance: Importance.high,
38          priority: Priority.high,
39        );
40      const NotificationDetails platformChannelSpecifics =
41        NotificationDetails(android: androidPlatformChannelSpecifics);
42
43      await flutterLocalNotificationsPlugin.show(
44        0, // Notification ID
45        'Gas Level Alert', // Title
46        'Gas level has reached $gaslevel ppm!', // Body
47        platformChannelSpecifics,
48      );
49    }
50  }
```

```
Gaslevel.dart
C:\Users\WIN10-1> AppData\Local\Temp\Rar$Dla15108.33634 > Gaslevel.dart
51 void _listenForGasLevelUpdates() {
52   FirebaseFirestore.instance
53     .collection('EspData')
54     .orderBy('timestamp', descending: true)
55     .limit(1)
56     .snapshots()
57     .listen((snapshot) {
58       if (snapshot.docs.isNotEmpty) {
59         Map<String, dynamic> data = snapshot.docs.first.data();
60         int gaslevel = (data['GasLevel'] as num).toInt();
61
62         if (gaslevel > 50) {
63           _showNotification(gaslevel);
64         }
65       }
66     });
67
68
69   @override
70   Widget build(BuildContext context) {
71     return Scaffold(
72       appBar: AppBar(
73         backgroundColor: Colors.black,
74         elevation: 0,
75         leading: IconButton(
76           icon: const Icon(Icons.arrow_back, color: Colors.white),
77           onPressed: () {
78             Navigator.pop(context);
79           },
80         ),
81       title: const Text(
82         'GAS LEVEL',
83         style: TextStyle(
84           color: Colors.white,
85           fontFamily: 'Cursive',
86           fontSize: 24,
87           fontWeight: FontWeight.bold,
88         ),
89       ),
90       centerTitle: true,
91     ),
92     body: StreamBuilder<QuerySnapshot>(
93       stream: FirebaseFirestore.instance
94         .collection('EspData')
95         .orderBy('timestamp', descending: true)
96         .limit(1)
97         .snapshots(),
98       builder: (context, snapshot) {
99       }
```

```

13 class _Gas_LevelState extends State<Gas_Level> {
14   Widget build(BuildContext context) {
15     return Scaffold(
16       body: StreamBuilder<QuerySnapshot>(
17         stream: FirebaseFirestore.instance
18         .collection('gas_level')
19         .snapshots(),
20       ),
21     );
22   }
23   if (snapshot.hasError) {
24     return Center(child: Text('Error: ${snapshot.error}'));
25   }
26   if (snapshot.connectionState == ConnectionState.waiting) {
27     return Center(child: CircularProgressIndicator());
28   }
29   if (!snapshot.hasData || snapshot.data!.docs.isEmpty) {
30     return Center(child: Text('No data available'));
31   }
32   Map<String, dynamic> data =
33   | snapshot.data!.docs.first.data() as Map<String, dynamic>;
34   int gasLevel = (data['gasLevel'] as num).toInt();
35   Color backgroundColor = gasLevel > 50 ? Colors.red : Colors.black;
36   Color textColor = gasLevel > 50 ? Colors.white : Colors.white70;
37   return Container(
38     color: backgroundColor,
39     child: Center(
40       child: FadeIn(
41         duration: const Duration(milliseconds: 500),
42         child: Container(
43           padding: const EdgeInsets.all(20),
44           decoration: BoxDecoration(
45             border: Border.all(
46               color: gasLevel > 50 ? Colors.red : Colors.blueAccent,
47               width: 4),
48             borderRadius: BorderRadius.circular(15),
49             gradient: LinearGradient(
50               colors: gasLevel > 50
51               ? [Colors.red.shade900, Colors.red.shade700]
52               : [
53                 Colors.blueGrey.shade900,
54                 Colors.blueGrey.shade700
55               ],
56               begin: Alignment.topLeft,
57               end: Alignment.bottomRight,
58             ),
59           ),
60       ),
61     ),
62   );
63 }

```

```

13 class _Gas_LevelState extends State<Gas_Level> {
14   Widget build(BuildContext context) {
15     return Scaffold(
16       body: StreamBuilder<QuerySnapshot>(
17         stream: FirebaseFirestore.instance
18         .collection('gas_level')
19         .snapshots(),
20       ),
21     );
22   }
23   if (snapshot.hasError) {
24     return Center(child: Text('Error: ${snapshot.error}'));
25   }
26   if (snapshot.connectionState == ConnectionState.waiting) {
27     return Center(child: CircularProgressIndicator());
28   }
29   if (!snapshot.hasData || snapshot.data!.docs.isEmpty) {
30     return Center(child: Text('No data available'));
31   }
32   Map<String, dynamic> data =
33   | snapshot.data!.docs.first.data() as Map<String, dynamic>;
34   int gasLevel = (data['gasLevel'] as num).toInt();
35   Color backgroundColor = gasLevel > 50 ? Colors.red : Colors.black;
36   Color textColor = gasLevel > 50 ? Colors.white : Colors.white70;
37   return Container(
38     color: backgroundColor,
39     child: Center(
40       child: FadeIn(
41         duration: const Duration(milliseconds: 500),
42         child: Container(
43           padding: const EdgeInsets.all(20),
44           decoration: BoxDecoration(
45             border: Border.all(
46               color: gasLevel > 50 ? Colors.red : Colors.blueAccent,
47               width: 4),
48             borderRadius: BorderRadius.circular(15),
49             gradient: LinearGradient(
50               colors: gasLevel > 50
51               ? [Colors.red.shade900, Colors.red.shade700]
52               : [
53                 Colors.blueGrey.shade900,
54                 Colors.blueGrey.shade700
55               ],
56               begin: Alignment.topLeft,
57               end: Alignment.bottomRight,
58             ),
59           ),
60       ),
61     ),
62   );
63 }

```

3.3 ARDUINO

In our project, we used the ESP32 microcontroller as the main component to enable communication between the hardware and backend services. The ESP32 was programmed to make HTTP requests to a PHP server (Melair) in order to perform specific tasks such as sending automated emails. Moreover, the ESP32 was responsible for sending real-time data to our mobile application through Firebase, allowing for live monitoring and synchronization. To enhance the connectivity and ensure reliable cloud communication, we also integrated IoT (Internet of Things Communication). This setup allowed us to create an efficient, smart, and connected system that serves the core functionality of our application.

Arduino

Arduino is an open-source electronics platform that is designed to make digital electronics more accessible. It consists of a physical programmable circuit board (commonly called a microcontroller) and a development environment (IDE) for writing and uploading code to the board. Arduino is widely used in DIY projects, robotics, smart devices, and educational purposes.

Main Components of the Arduino Uno Board (as shown in the image):

Microcontroller (ATmega328P):

This is the "brain" of the Arduino. It processes inputs and controls outputs according to the uploaded program.

1. Digital Pins (0-13):

These can be used as input or output pins. Some of them support PWM (Pulse Width Modulation), marked with ~.

2. Analog Input Pins (A0-A5):

These pins read analog signals from sensors (like temperature or light sensors) and convert them to digital values.

3. Power Pins:

- **3.3V**: Provides 3.3 volts of power.
- **5V**: Provides 5 volts of power (used to power most components).
- **GND (Ground)**: Completes the circuit by connecting to the ground.
- **Vin**: Voltage input when using an external power source.

4. USB Port:

Used to upload code from a computer and provide power.

5. Barrel Jack:

For supplying external power (7–12V) if USB is not used.

6. Reset Button:

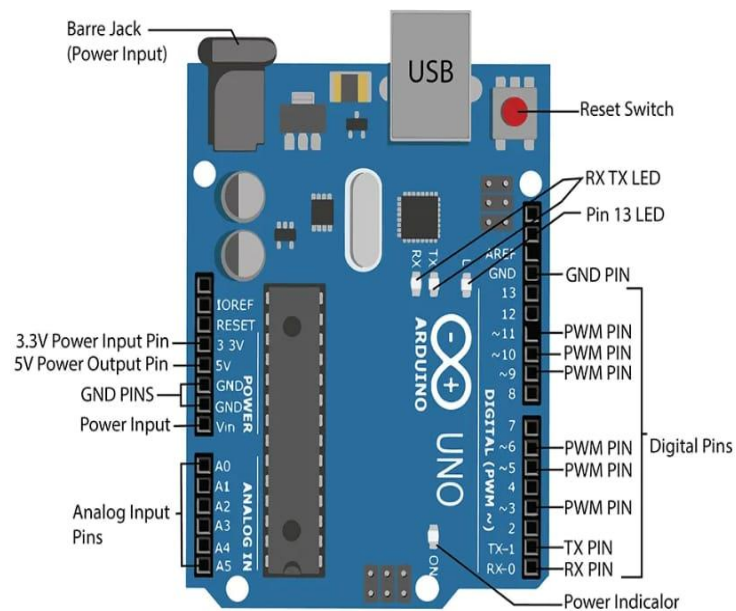
Restarts the code running on the board.

7. LED Indicators:

- **Power LED**: Lights up when the board is powered.
- **Pin 13 LED**: Connected to digital pin 13, useful for testing.
- **RX/TX LEDs**: Indicate data being transmitted/received via serial communication.

Uses of Arduino:

- Building robots and automation systems.
- Creating smart home devices (e.g., automatic lights or alarms).
- Collecting sensor data (e.g., temperature, humidity).
- Learning programming and electronics in a fun and interactive way.
- Projects involving Internet of Things (IoT).



ARDUINO

ESP32

The ESP32 is a low-cost, low-power system-on-chip (SoC) microcontroller developed by Espressif Systems. It is the successor to the popular ESP8266 and comes with more powerful features. The ESP32 has built-in Wi-Fi and Bluetooth (Classic and BLE) capabilities, making it ideal for Internet of Things (IoT) applications.

Main Features of ESP32:

- Dual-core 32-bit processor (up to 240 MHz)
- Wi-Fi and Bluetooth integrated on-chip
- Up to 34 GPIO pins
- Multiple ADC (Analog-to-Digital Converter) channels
- Two DAC (Digital-to-Analog Converter) channels
- Support for PWM, SPI, I2C, I2S, UART
- Touch-sensitive input pins
- Deep sleep mode for low power consumption
- Can be programmed using Arduino IDE, MicroPython, or ESP-IDF

Common Uses of ESP32:

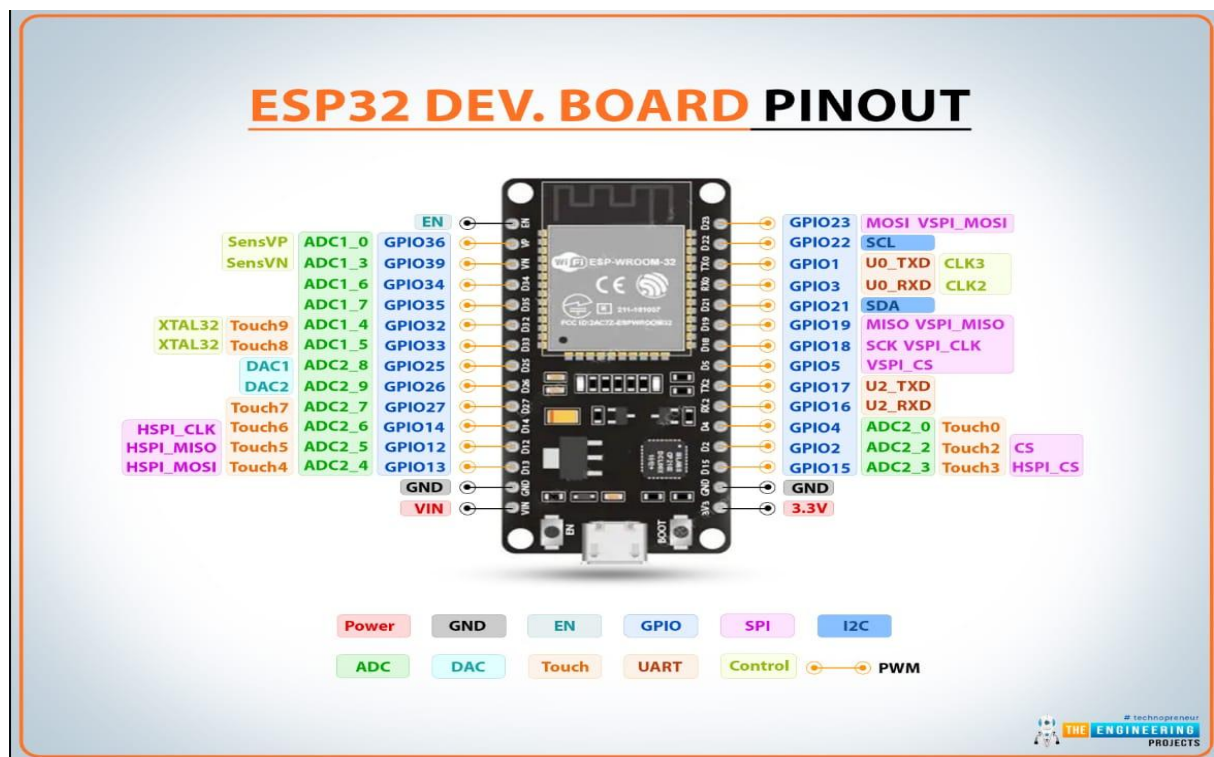
- Home automation (smart lights, security systems, sensors)
- Wireless sensor networks
- IoT devices that communicate over Wi-Fi or Bluetooth
- Robotics and remote control systems
- Wearables and portable electronics
- Projects needing real-time data monitoring and cloud communication

ESP32 Pinout Overview (based on the image):

- **Power Pins:** VIN, 3.3V, GND
- **ADC (Analog Input):** Multiple ADC1 and ADC2 channels
- **DAC:** DAC1 and DAC2 pins
- **Touch Pins:** Can detect capacitive touch (e.g., GPIO4 = Touch0)
- **SPI Pins:** HSPI_CLK, HSPI_MISO, HSPI_MOSI, etc.
- **I2C Pins:** SDA and SCL
- **UART:** U0_TXD, U0_RXD (default serial), U2_TXD, U2_RXD
- **PWM:** Most GPIOs support PWM output
- **GPIO Pins:** General-purpose pins for digital input/output

Why Use ESP32?

- More powerful and versatile than many Arduino boards
- Built-in Wi-Fi/Bluetooth eliminates need for external modules
- Excellent for wireless communication and real-time processing
- Affordable and supported by a large community



EPS32

A Driver

A driver in electronics is a piece of software or hardware that allows a microcontroller (like Arduino or ESP32) to communicate with and control other hardware components. Drivers are especially important when dealing with motors, displays, sensors, or external devices that need special signals or protocols.

A driver acts like a bridge between a microcontroller and other hardware components. Without drivers, many electronic devices would not be able to function properly or communicate correctly.

Types of Drivers:

1. Software Driver:

A software program or library that helps the microcontroller understand how to communicate with a device.

Example: The LiquidCrystal library in Arduino is a software driver for LCD displays.

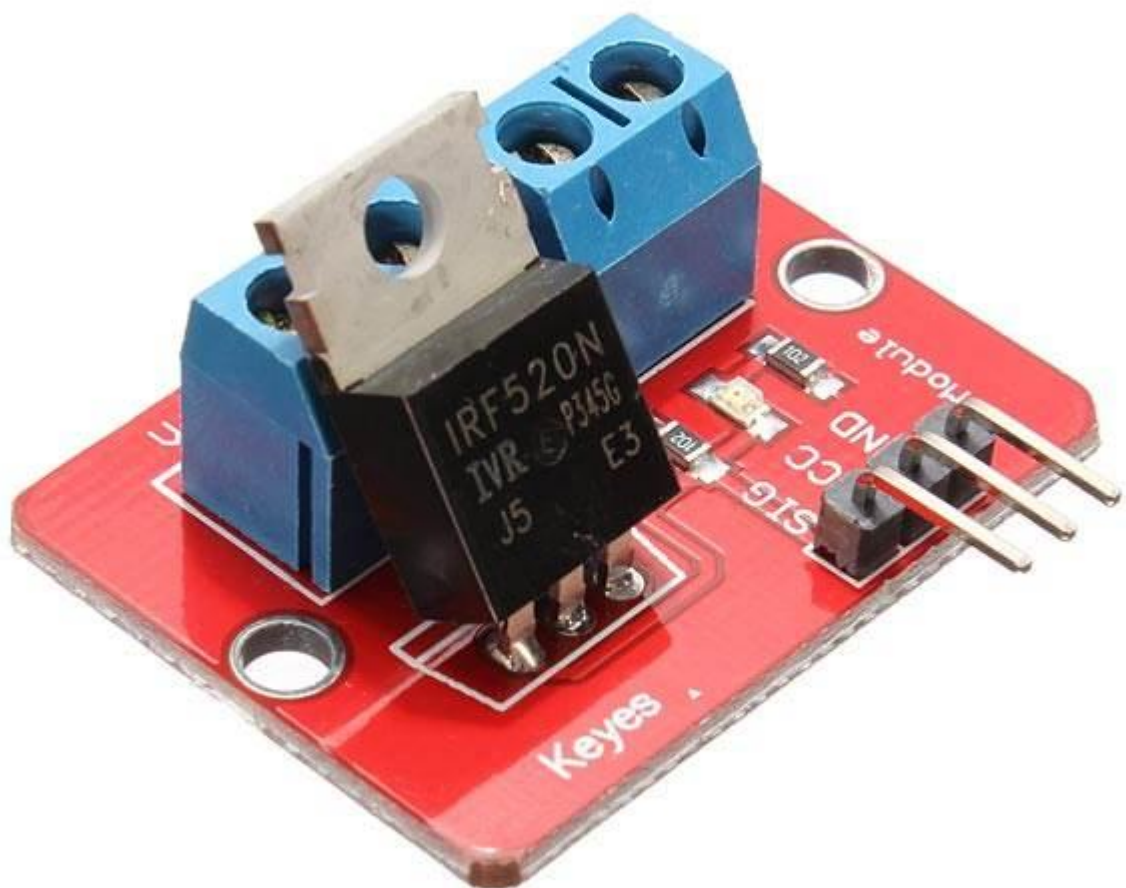
2. Hardware Driver:

A physical electronic component (like an IC or circuit) used to drive or control another device.

Example: The L298N motor driver is a hardware module that allows the Arduino to control DC motors.

Why Drivers are Important:

- They help manage the power requirements of high-power devices (like motors).
- They protect the microcontroller from overload.
- They translate digital signals into the appropriate control signals for the device.
- They simplify communication with complex modules like sensors, displays, and communication chips.



DRIVER

Gas Sensor

A gas sensor (also called a gas detector) is an electronic device that detects the presence of gases in the air. It is commonly used to monitor gas leaks, measure air quality, and ensure safety in homes, industries, and laboratories.

Gas sensors are vital for safety, environmental monitoring, and smart systems. They are easy to integrate with platforms like Arduino or ESP32 and provide essential real-time information about air and gas conditions.

How to Use a Gas Sensor with Arduino or ESP32 ?

1. Connect the sensor's VCC to 5V (or 3.3V if supported), GND to ground.
2. Connect the analog output (A0) or digital output (D0) to a microcontroller pin.
3. Read the values from the sensor using code.
4. Take actions like turning on an alarm or sending a notification if gas is detected.

How Does a Gas Sensor Work ?

Gas sensors work by detecting specific gases through a chemical reaction or change in electrical properties (like resistance) when exposed to certain gases. The sensor outputs a voltage or signal that can be read by a microcontroller like Arduino or ESP32.



Gas Sensor

A heater

A heater in electronics is a device that converts electrical energy into heat. It is used in many applications where controlled heating is required. In embedded systems and Arduino projects, heaters are used in devices like temperature control systems, incubators, 3D printers, or gas sensors (like MQ series) which require a built-in heater to function.

A heaters are essential components in many electronic and smart systems that need temperature control. When used with Arduino or ESP32, they enable automation in heating processes in a safe and efficient way.

Types of Heaters in Electronics:

1. Ceramic Heaters:

- Used in 3D printers and space heaters.
- Provide fast and stable heating.

2. Nichrome Wire:

- A resistive wire that heats up when current flows through it.
- Commonly used in homemade heaters or heating pads.

3. PTC Heaters (Positive Temperature Coefficient):

- Self-regulating heaters that stop heating at a certain temperature.
- Safe and efficient for small devices.

4. Heating Element in Gas Sensors:

- Sensors like MQ-2, MQ-3, MQ-135 have a small built-in heater to heat the sensing element for accurate gas detection.

Applications of Heaters:

- Temperature-controlled projects (with Arduino/ESP32 + relay + heater)
- Incubators (for eggs or biological experiments)
- 3D printers (to heat the nozzle and print bed)
- Humidity controllers
- Gas sensors (to burn and detect gases).

How to Control a Heater with Arduino or ESP32:

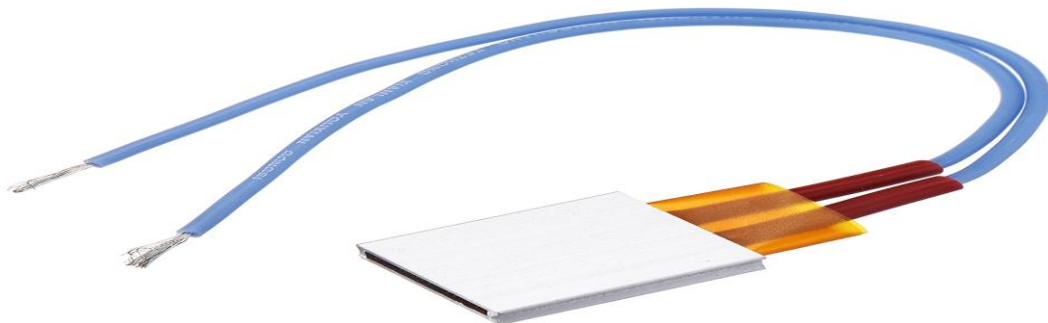
Since heaters usually operate on high voltage or high current, they are controlled using relays or MOSFETs.

Example Components:

- Arduino/ESP32
- Relay module
- Heater (resistive)
- Temperature sensor (like DHT11 or DS18B20)

Example Logic:

- Read temperature from sensor.
- If temperature is below set value, turn on the relay to activate the heater.
- If temperature is above set value, turn off the relay.



Heaters

Install The Application



https://www.mediafire.com/file/6c0vygeqv65txba/Gas_v_5.apk/file

CONCLUSION

This smart gas leak detection and response system represents an innovative approach to addressing one of the most dangerous household hazards. By combining advanced sensor technology with automated safety mechanisms and intelligent notification systems, the project creates a comprehensive solution for gas leak prevention and emergency response. The system's ability to detect leaks at their earliest stages and initiate multiple protective actions simultaneously significantly enhances safety standards for water heater installations in both residential and commercial settings.

The integration of real-time monitoring with mobile connectivity ensures users maintain awareness of their system's status regardless of location, while the automatic response features provide crucial protection even when occupants are absent. The project successfully bridges the gap between physical safety systems and digital monitoring capabilities, creating a robust defense against potential gas-related accidents.

Looking forward, this technology establishes a foundation for further development in smart safety systems, with potential applications extending beyond water heaters to comprehensive gas monitoring solutions for entire properties. Its implementation promises to reduce gas-related accidents, protect property, and most importantly, save lives by preventing situations that could lead to explosions or health hazards. The system stands as a testament to how intelligent technology can be harnessed to create safer living and working environments for all.

REFERENCES

1. **Firestore Documentation.** (n.d.). *Realtime Database*. Retrieved from <https://firebase.google.com/docs/database>
2. **Espressif Systems.** (n.d.). *ESP32 Series Datasheet*. Retrieved from <https://www.espressif.com/en/products/socs/esp32/resources>...Official documentation of the ESP32 microcontroller, including Wi-Fi, I2C, and HTTP communication capabilities.
3. **Arduino Official Website.** (n.d.). *Arduino Uno Rev3*. Retrieved from <https://store.arduino.cc/products/arduino-uno-rev3> ..Provides specifications and capabilities of the Arduino Uno, including sensor integration.
4. **Grinberg, M.** (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media...Covers building web applications and REST APIs using Flask, including integration with AI/ML models. **Ibrahim, D.,** (2019). *Internet of Things with ESP8266: Build IoT Projects Using the ESP8266*. Elektor International Media.
5. Though focused on ESP8266, it discusses IoT systems architecture relevant to ESP32 applications.
6. **Sethi, P., & Sarangi, S. R.** (2017). *Internet of Things: Architectures, Protocols, and Applications*. *Journal of Electrical and Computer Engineering*, 2017. <https://doi.org/10.1155/2017/9324035>
7. Hussain, A., Mkpojiogu, E. O. C., & Yusof, M. M. (2016). *Usability evaluation of mobile applications: A systematic review*. *Indian Journal of Science and Technology*, 9(37). <https://doi.org/10.17485/ijst/2016/v9i37/102961>
8. W3Schools. (n.d.). *HTML Introduction*. Retrieved from https://www.w3schools.com/html/html_intro.asp
9. Mozilla Developer Network (MDN). (n.d.). *HTML: HyperText Markup Language*. Retrieved from <https://developer.mozilla.org/en-US/docs/Web/HTML>
10. GeeksforGeeks. (2023). *Why is HTML Important?*. Retrieved from <https://www.geeksforgeeks.org/why-is-html-important/>
11. Sharma, V. (2020). *HTML for Beginners: Learn HTML in Easy Way*. Amazon Kindle Direct Publishing.
12. Duckett, J. (2011). *HTML and CSS: Design and Build Websites*. Wiley.
13. **Freeman, E., & Robson, E.** (2014). *Head First HTML and CSS* (2nd ed.). O'Reilly Media.
14. **Ullman, L.** (2017). *PHP for the Web: Visual QuickStart Guide* (5th ed.). Peachpit Press.. ISBN: 9780134291253.. pp. 1–10& pp. 27–30
15. **Duckett, J.** (2014). *JavaScript and JQuery: Interactive Front-End Web Development*. Wiley..ISBN: 9781118531648.. pp. 10–25& pp. 44–55
16. **Stroustrup, B.** (2013). *The C++ Programming Language* (4th ed.). Addison-Wesley...ISBN: 9780321563842...pp. 1–20& pp. 25–30
17. **Alessandro Biessek** (2019). *Beginning Flutter: A Hands-On Guide to App Development*. Apress...ISBN: 978-1484249718... pp. 1–15, pp. 29–40
18. **Zelle, J. M.** (2017). *Python Programming: An Introduction to Computer Science* (3rd ed.). Franklin, Beedle & Associates Inc.ISBN:9781590282755.. pp. 1–12& pp. 22–35



كلية العلوم
قسم الرياضيات

نموذج تقييم مشروع التخرج (مادة البحث والمقال)

اسم الطالب /

الفرقة الرابعة شعبة /

أولاً : تقييم المشرف على المشروع

بنود التقييم	درجة القصوى	درجة الطالب
تسليم البحث في الموعد المحدد	٥	
اكتمال العناصر الأساسية للمشروع	٥	
مواظبة الطالب على الاجتماع مع المشرف مرة أسبوعياً على الأقل	٥	
تجميع الموارد العلمية الكافية كما وكيفاً	٥	
المجموع	٢٠	

اسم المشرف / التوقيع /

ثانياً : تقييم المحكمين على المشروع

بنود التقييم	الدرجة القصوى	درجة الطالب
ملائمة أدوات البحث وجمع المعلومات وتحليلها	٥	
مدى كفاية البيانات التي تم جمعها للمشروع	٥	
التنسيق العام وجودة الشكل الظاهري للمخرج النهائي	٥	
سلامة اللغة ودقة التعبير	٥	
المنافسة الشفوية للطالب	٥	
عرض الطالب للمشروع	٥	
المجموع	٣٠	

الدرجة القصوى	درجة الطالب
٥٠	
المجموع الكلي لدرجات المشروع	

السادة المحكمين

أ.د. / التوقيع /
أ.د. / التوقيع /
أ.د. / التوقيع /

رئيس مجلس القسم